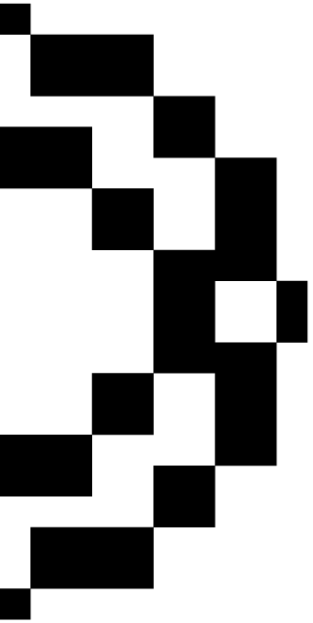
NOVA IMS INFORMATION MANAGEMENT SCHOOL

Sequential Models

Text Mining 2025-2026

Bruno Jardim
bjardim@novaims.unl.pt

Lecture Plan



01. Review of Word Embeddings

02. Recurrent Neural Networks

03. Long Short-Term Memory Neural Networks

04. Sequence-to-Sequence Models

01.
Review of Word Embeddings

Recall Problems with Bag of Words

Word Representations

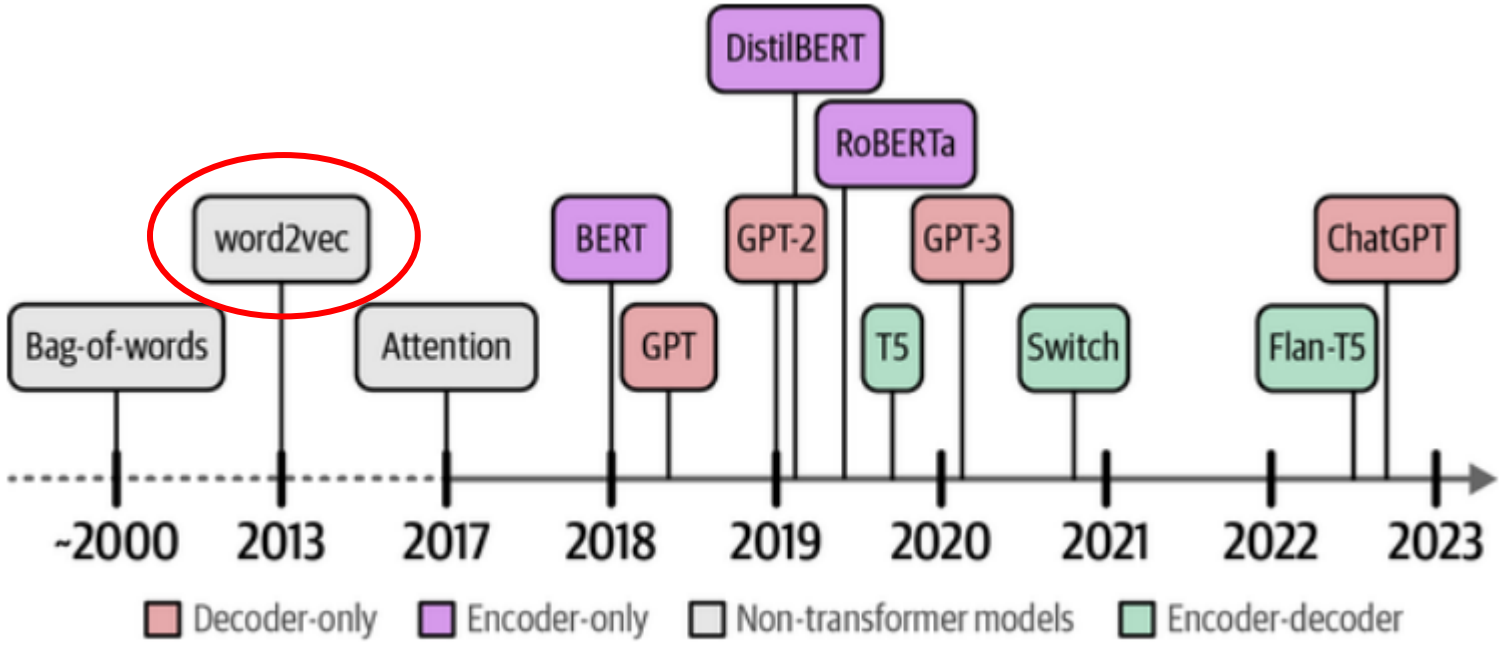
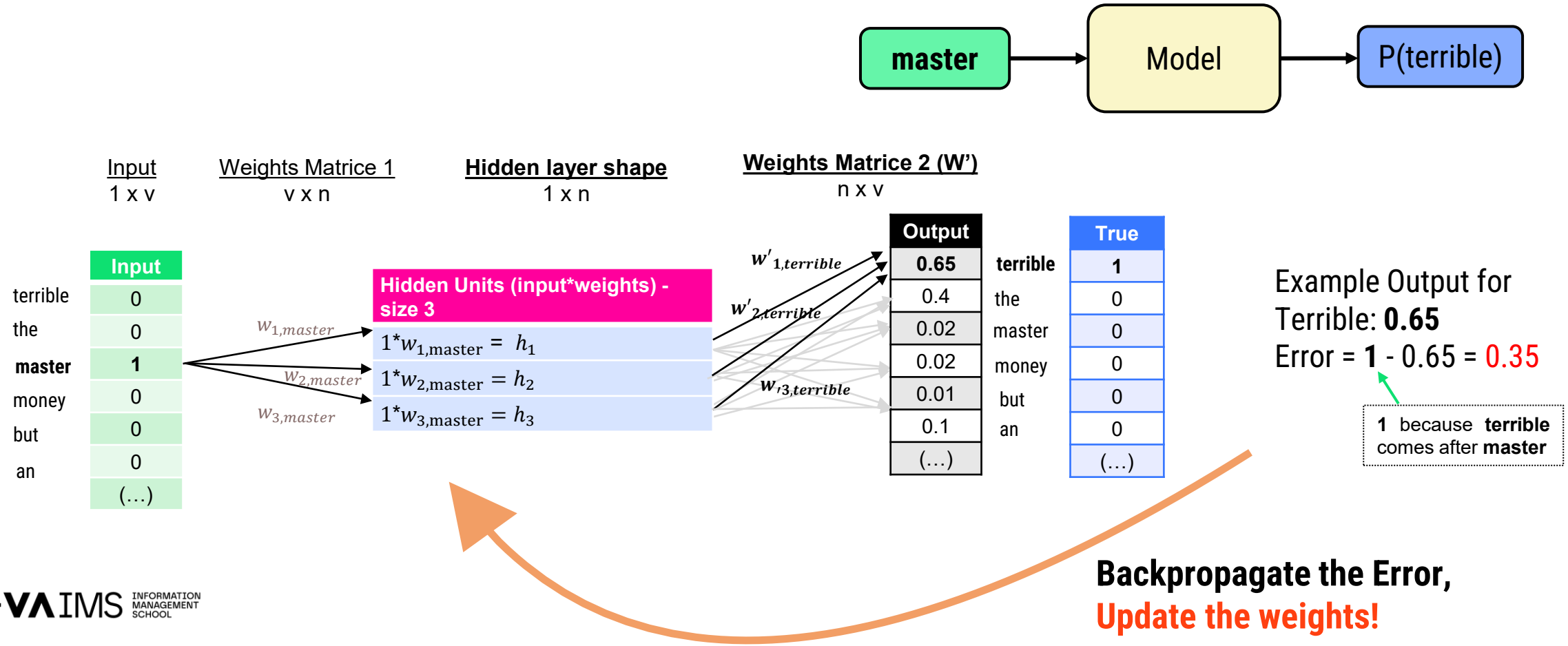


Figure 1-1. A peek into the history of Language AI.

Alammar, J., & Grootendorst, M. (2024). **Hands-On large language models: Language Understanding and Generation**. Sebastopol : O'Reilly Media.

Word2Vec (Skip-gram)

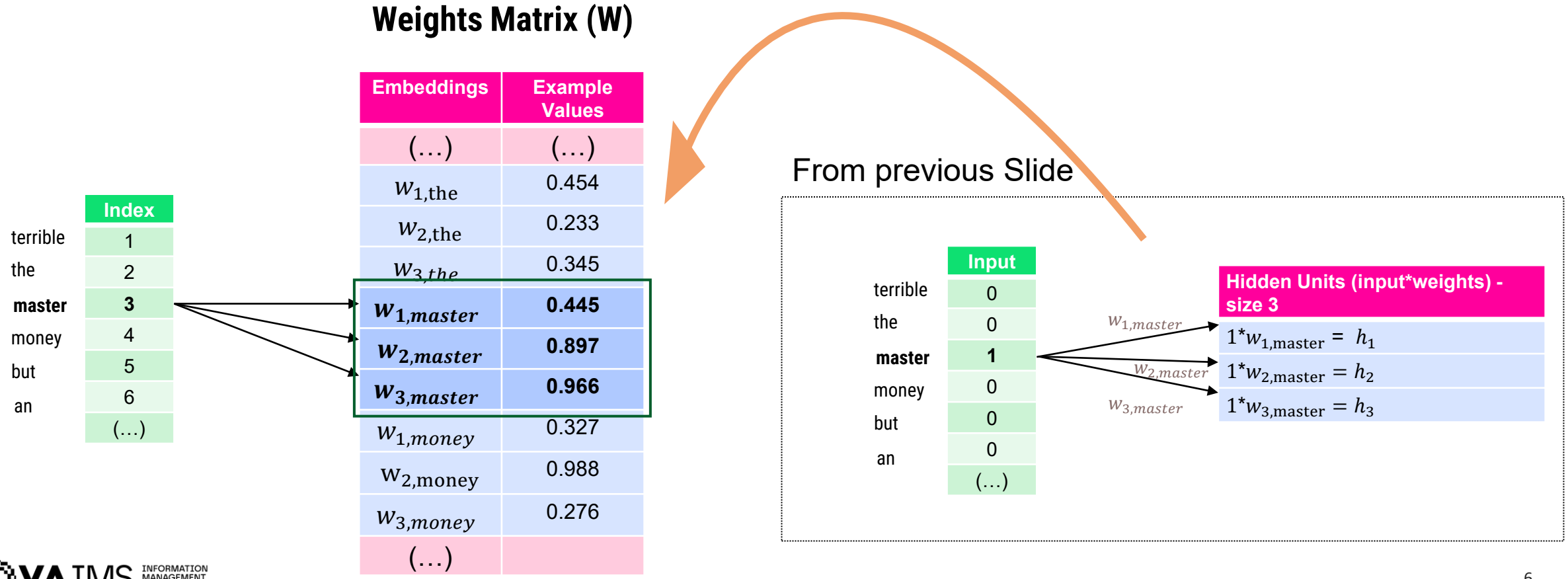
Word2Vec learns dense vector representations of words by training a shallow neural model to predict a target word from surrounding words (Skip-gram). Words used in similar contexts end up with similar vectors, enabling semantic relationships such as similarity and analogies.



After training the model, the **embeddings are stored in the Weights Matrix (W)** of the network

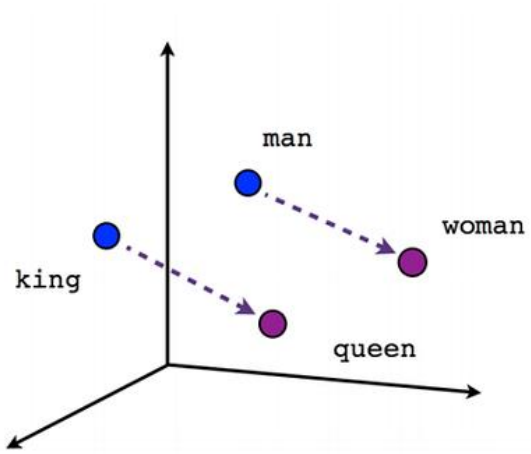
In the example below, **Master** can be represented as low dimensionality vector (an embedding):

[0.445, 0.897, 0.966]

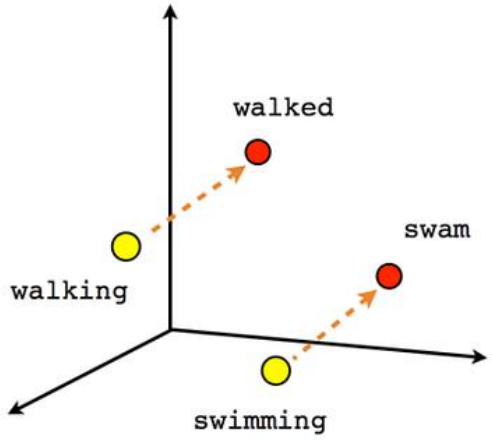


Word Embeddings Properties

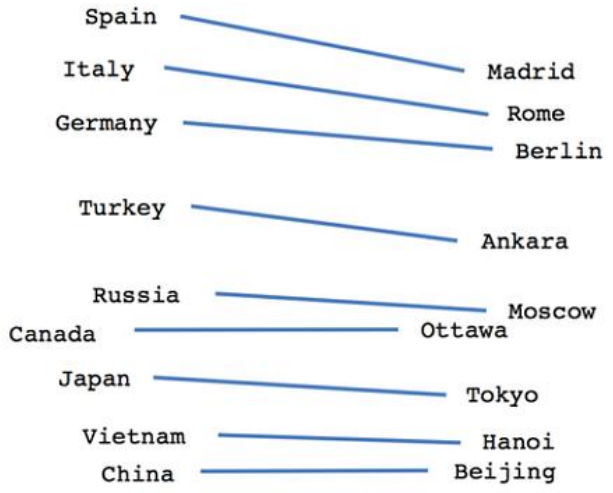
After trained, Word Embeddings can extract **semantic relationships** between words



Male-Female



Verb tense



Country-Capital

https://medium.com/data-science/creating-word-embeddings-coding-the-word2vec-algorithm-in-python-using-deep-learning-b337d0ba17a8

Recall how to input a BoW into a Classifier

Classification with Word2Vec

Recall how to input **BoW** vectors to model:

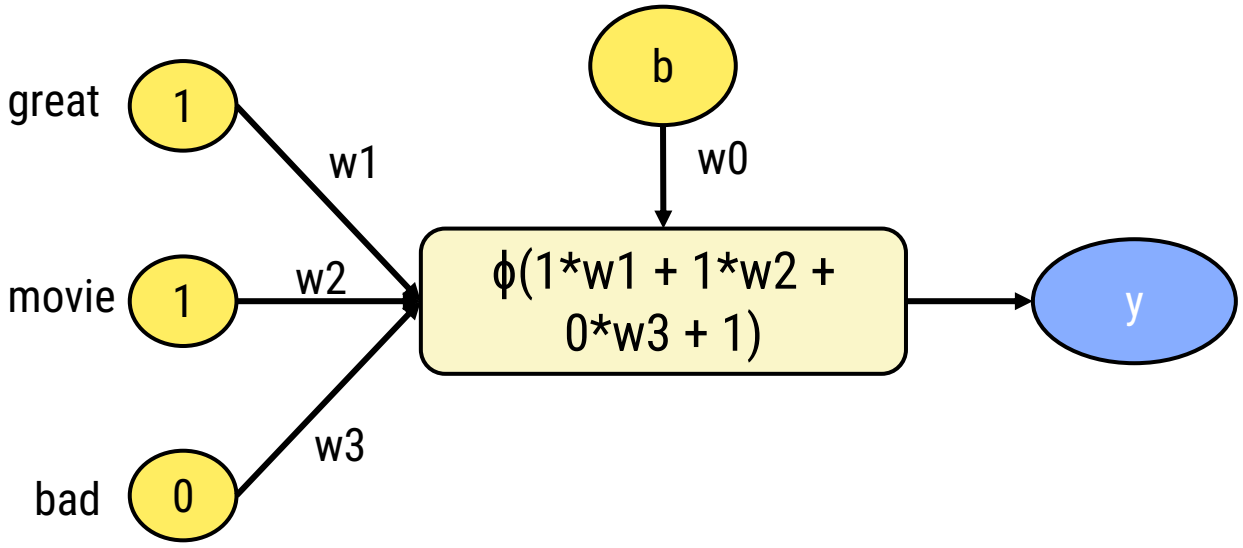
Review 1 – “Great movie”
Review 2 – “Bad movie”

	great	movie	bad
R1	**1**	**1**	**0**
R2	0	1	1

Task:

Predict the sentiment of movie reviews using a neural network.

Review 1:



Using Word2Vec as input to a Classifier

Classification with Word2Vec

What if we consider **word embeddings**:

Review 1 – “Great movie”
Review 2 – “Bad movie”

	great	movie	bad
R1	[0.034, 0.235, 0.987]	[0.765, 0.523, 0.895]	-
R2	-	[0.765, 0.523, 0.895]	[0.145, 0.534, 0.556]

Note: We assume embeddings size 3 for simplicity

Task:

Predict the sentiment of movie reviews using a neural network.

How does a sentence enter the network if it's a set of vectors and not a single vector?

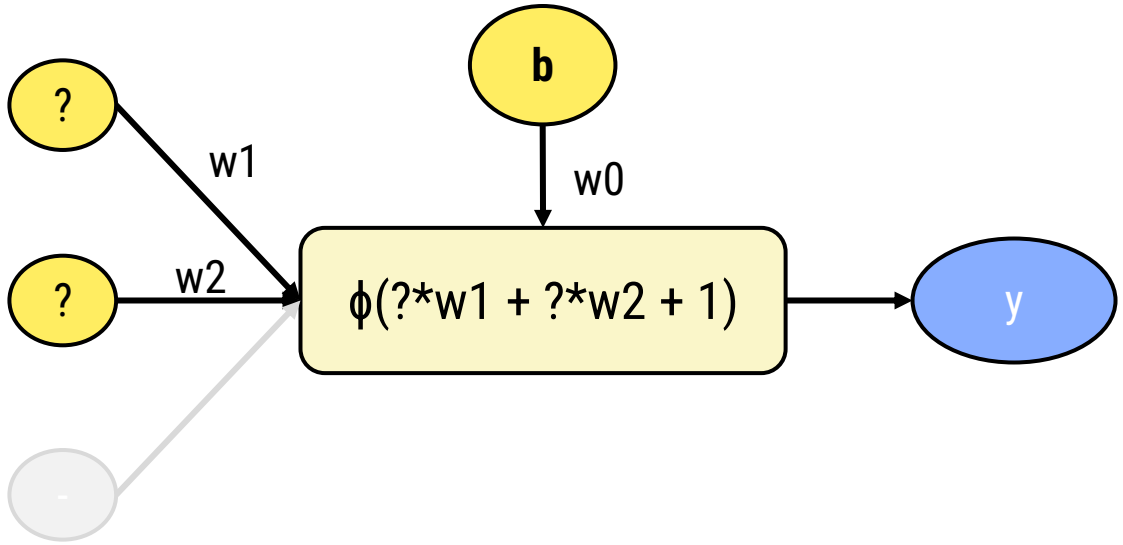
Review 1 – "Great movie"

	great	movie	bad
R1	**[0.034, 0.235, 0.987]**	**[0.765, 0.523, 0.895]**	**-**
R2	-	[0.765, 0.523, 0.895]	[0.145, 0.534, 0.556]

great=[0.034, 0.235, 0.987]

movie = [0.765, 0.523, 0.895]

Review 1:



One way could be to **average the vectors** of the sentence:

Review 1 – "Great movie"

(great+movie)/2 =

= ([0.034, 0.235, 0.987] + [0.765, 0.523, 0.895]) / 2 =

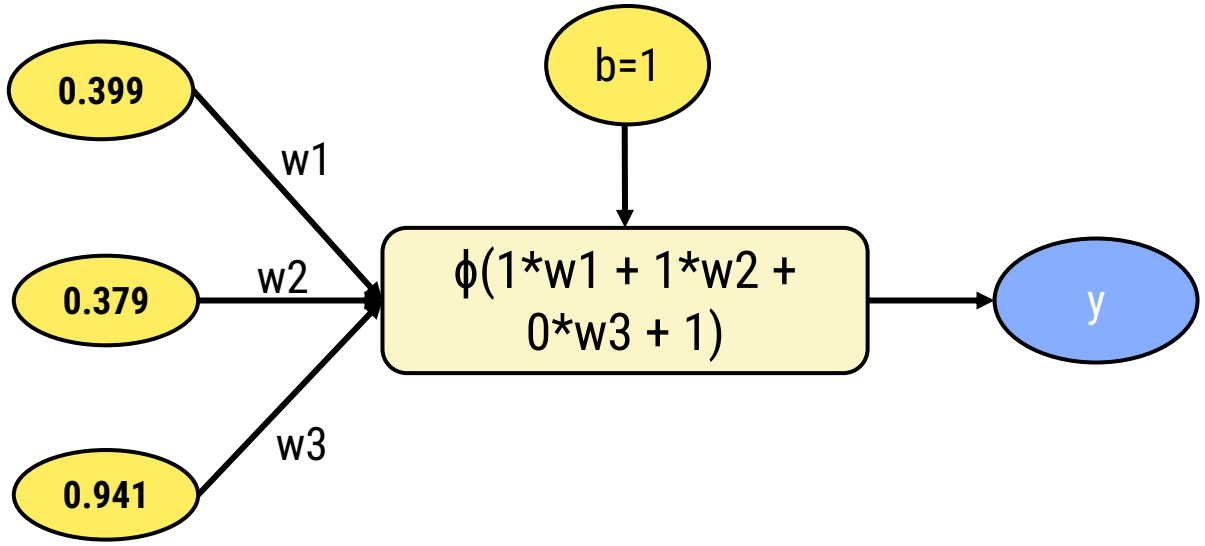
=([0.799, 0.758, 1.882]) / 2 =

=[0.399, 0.379, 0.941]

Now we have again an input able to enter the model:

Review 1 – “Great movie”

R1	0.399	0.379	0.941



BoW vs Word Embeddings

Sparse vs Dense vectors!

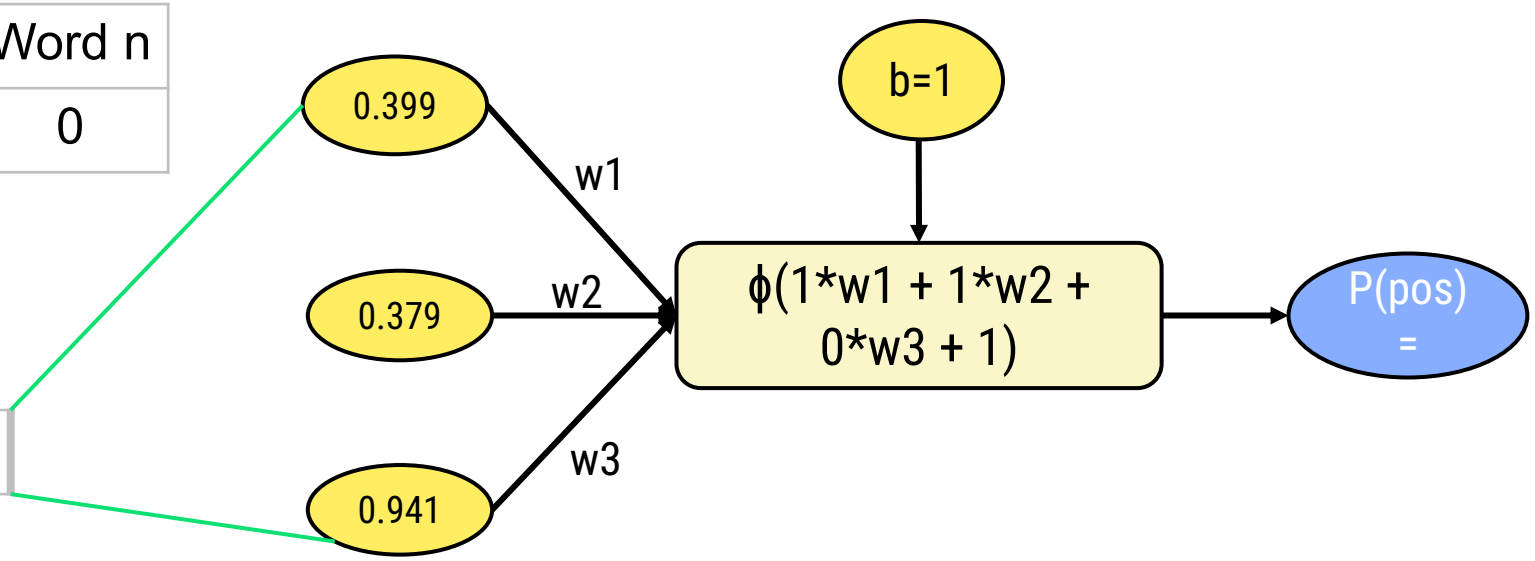
Review 1 – "Great movie"

Bag-of-Words

	great	movie	bad	…	Word n
R1	1	1	0	…	0

Word Embeddings

R1	**0.399**	**0.379**	**0.941**



What if we do not want to average/sum/concatenate the vectors?

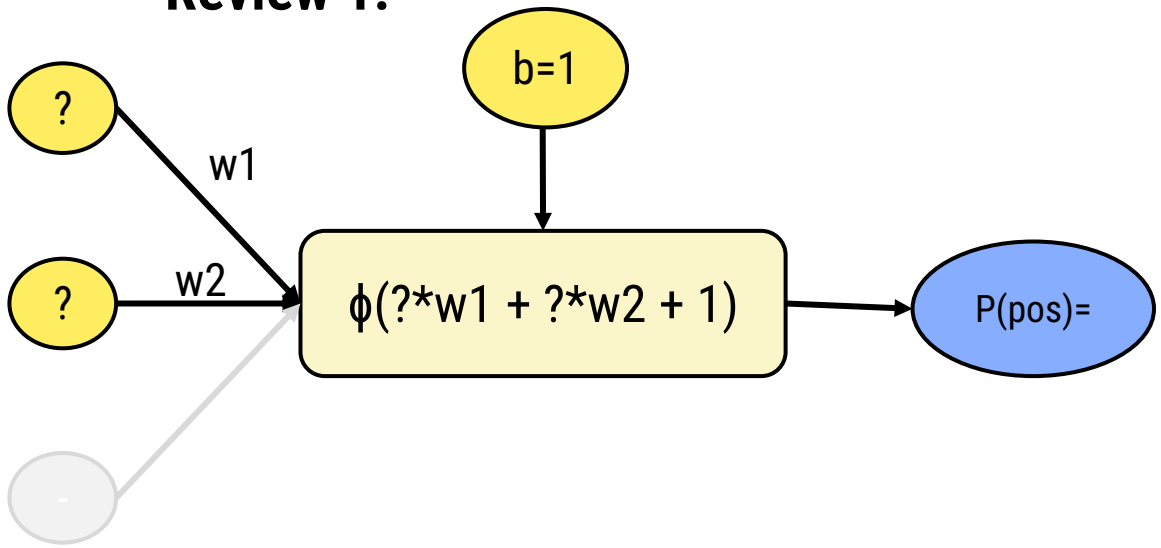
Task:

Predict the sentiment of movie reviews using a neural network.

Review 1:

great=[0.034, 0.235, 0.987]

movie = [0.765, 0.523, 0.895]



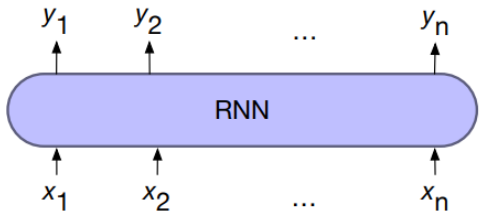
02.
Recurrent Neural Networks

To process sequential input, we can use some well know sequential models, like **Recurrent Neural Networks (RNN)** and **Long Short Term Memory Neural Networks (LSTM)**

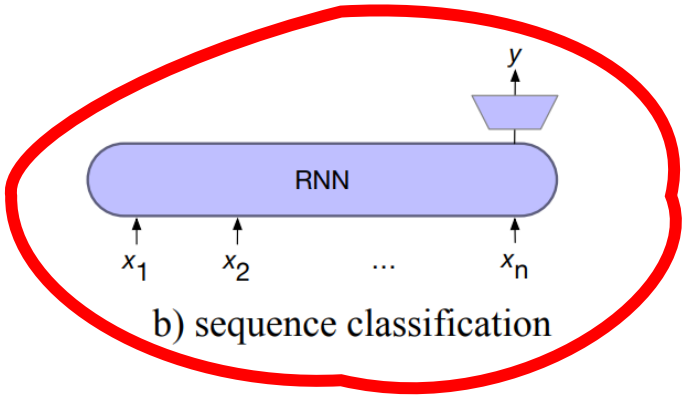
Common Tasks

- Next word prediction
- Sequence Labeling
- **Sequence Classification**
- Machine Translation

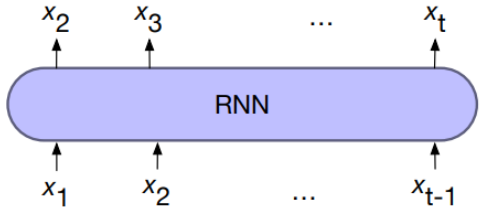
16



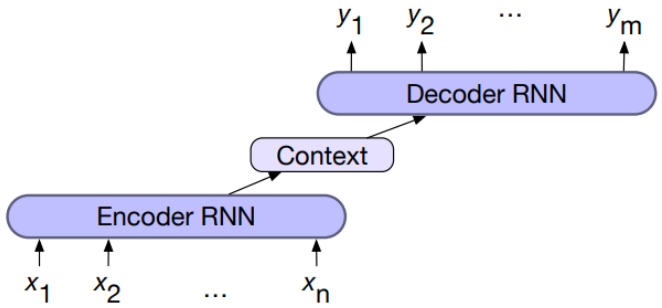
a) sequence labeling



b) sequence classification



c) language modeling



d) encoder-decoder

A recurrent neural network (RNN) is a network that contains a cycle within its network connections, meaning that **the value of some unit is directly, or indirectly, dependent on its own earlier outputs** as an input.

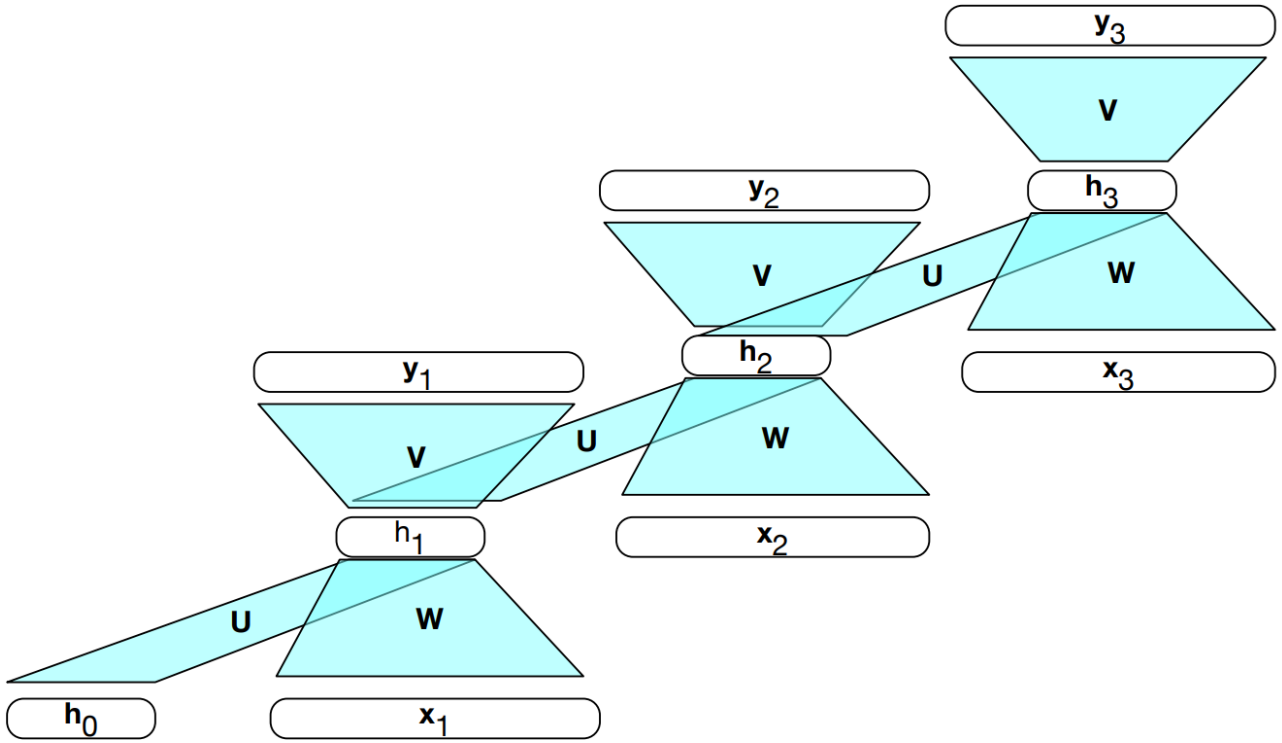
$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$
$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$$

The matrices U, V and W are shared across time, while new values for h and y are calculated with each time step

function FORWARDRNN($\mathbf{x}$, *network*) **returns** output sequence $\mathbf{y}$

$\mathbf{h}_0 \leftarrow 0$
for $i \leftarrow 1$ **to** LENGTH($\mathbf{x}$) **do**
  $\mathbf{h}_i \leftarrow g(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{x}_i)$
  $\mathbf{y}_i \leftarrow f(\mathbf{V}\mathbf{h}_i)$
return y

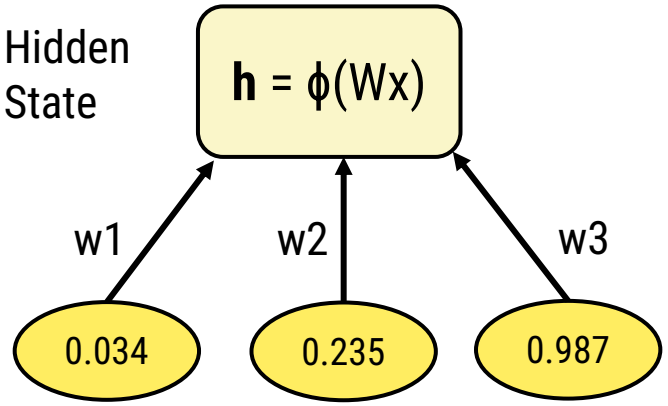
The matrices U, V and W are shared across time, while new values for h and y are calculated with each time step



Example:

Review 1 – "**Great** movie"

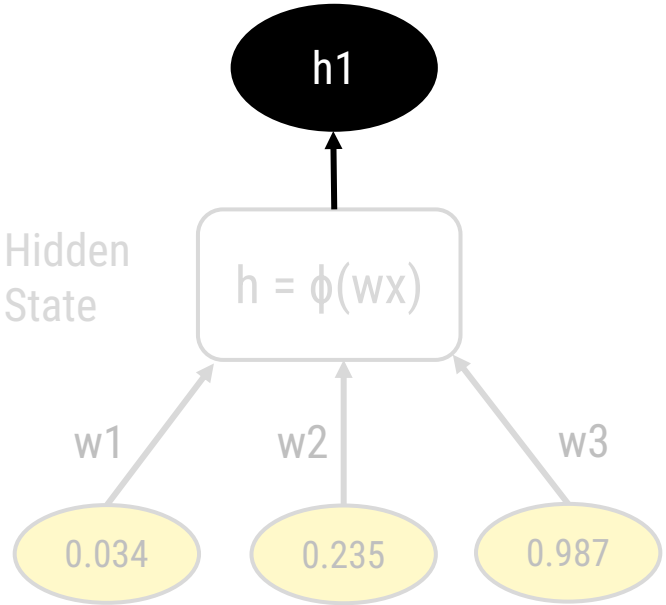
	Word Embeddings (size 3)	
	great	movie
R1	**[0.034, 0.235, 0.987]**	[0.765, 0.523, 0.895]



Example:

Review 1 – "**Great** movie"

	Word Embeddings (size 3)	
	great	movie
R1	**[0.034, 0.235, 0.987]**	[0.765, 0.523, 0.895]

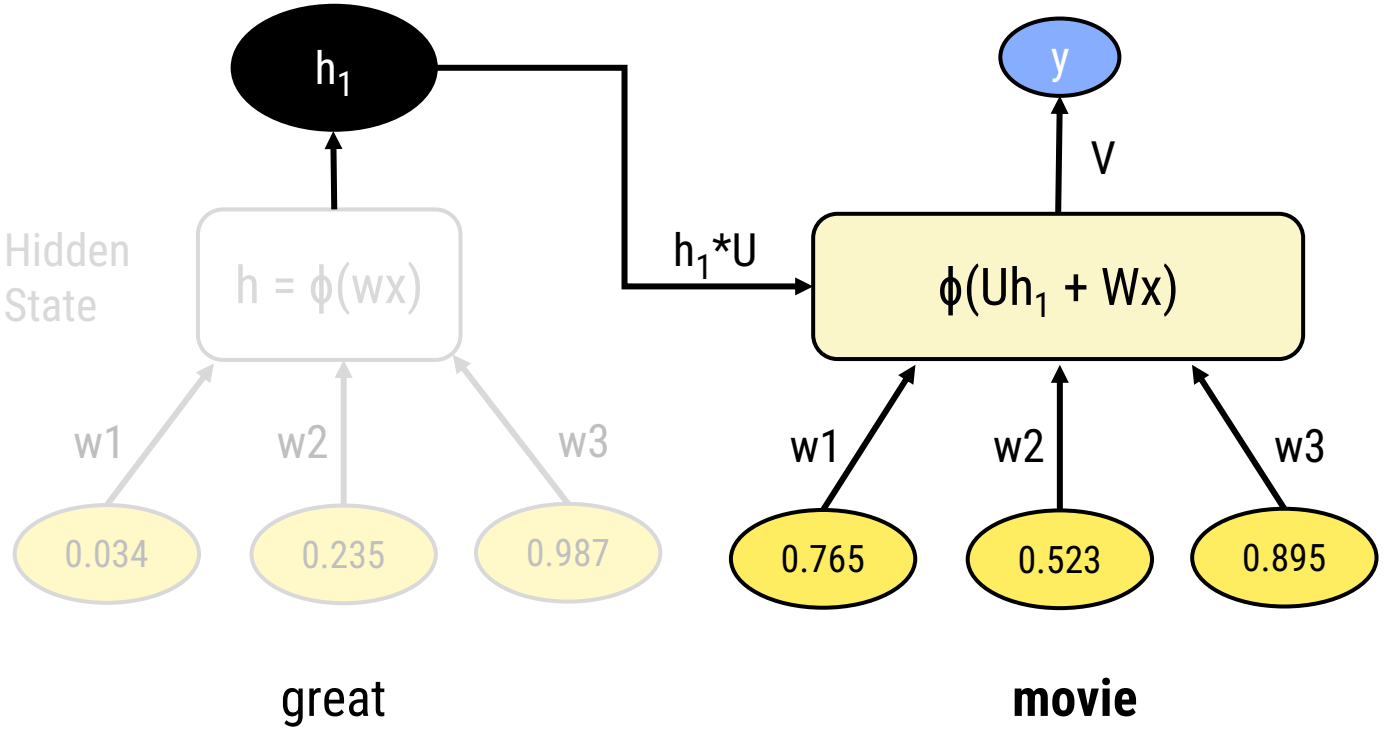


great

Example:

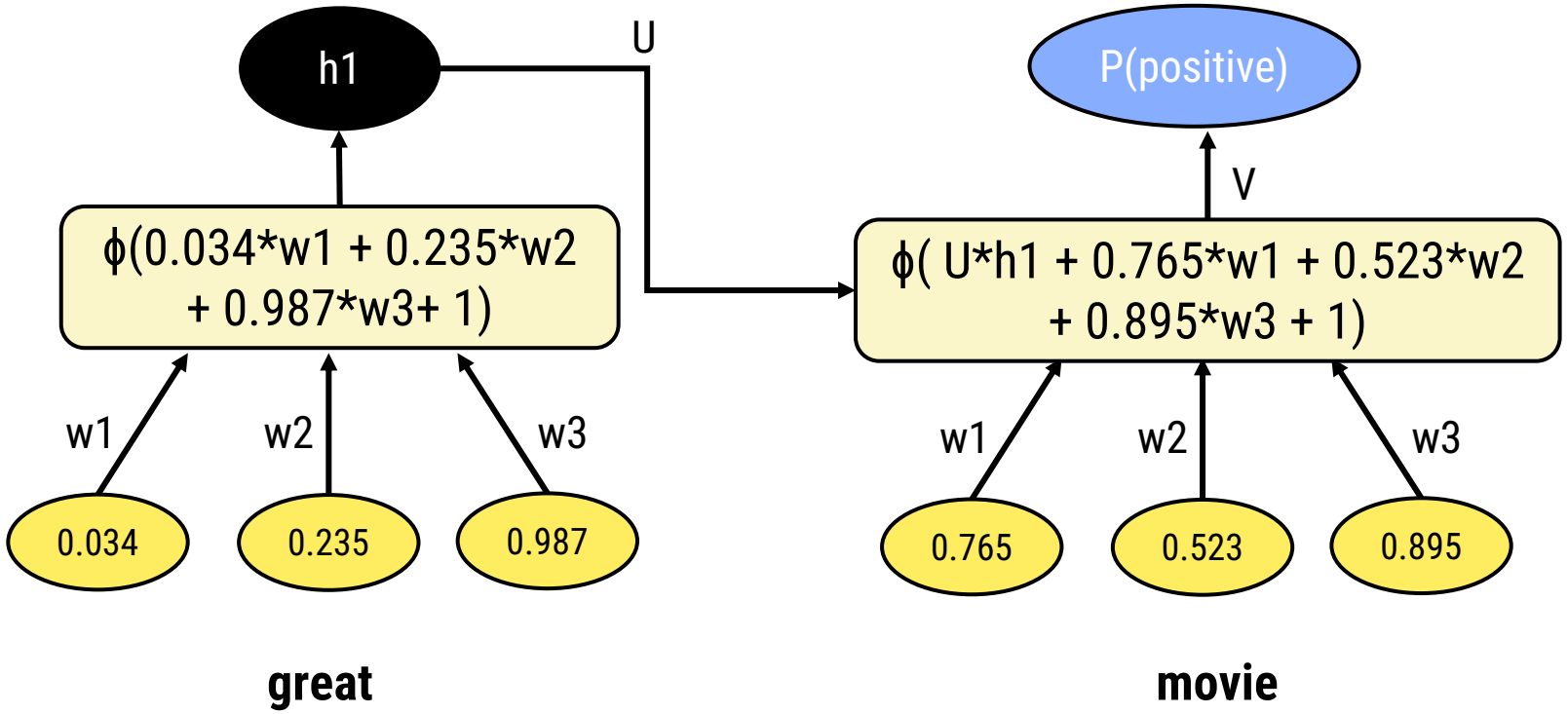
Review 1 – "Great **movie**"

	Word Embeddings (size 3)	
	great	**movie**
R1	[0.034, 0.235, 0.987]	**[0.765, 0.523, 0.895]**

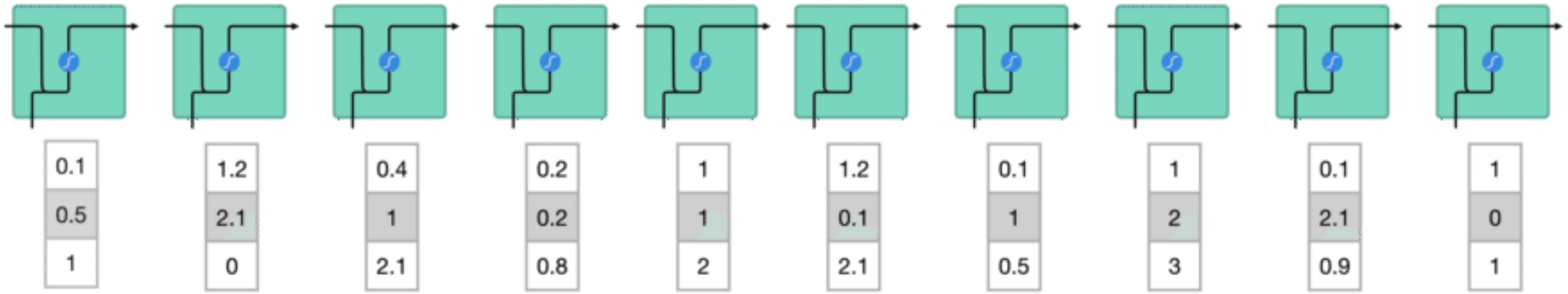


Example:

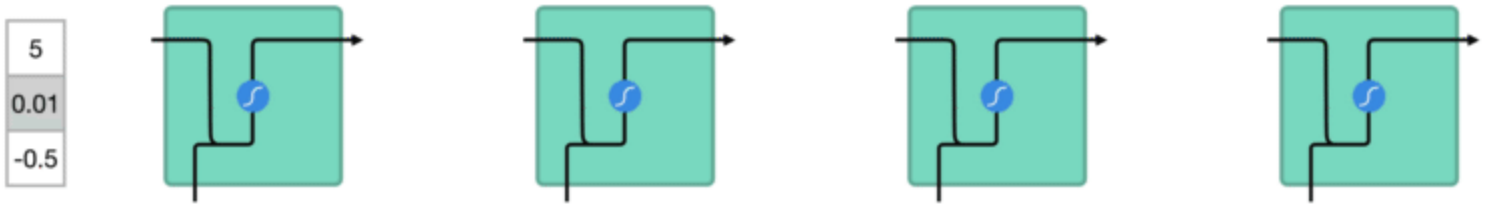
Review 1 – “Great movie”



For a sentence/document, **each word (embedding) is processed through the network one step at the time**:



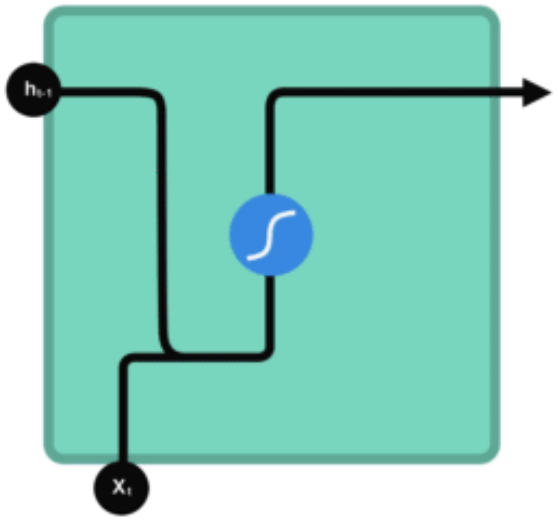
In parallel, the **hidden vector of the previous word, moves in the network** and is used to calculate the hidden vector of the next word

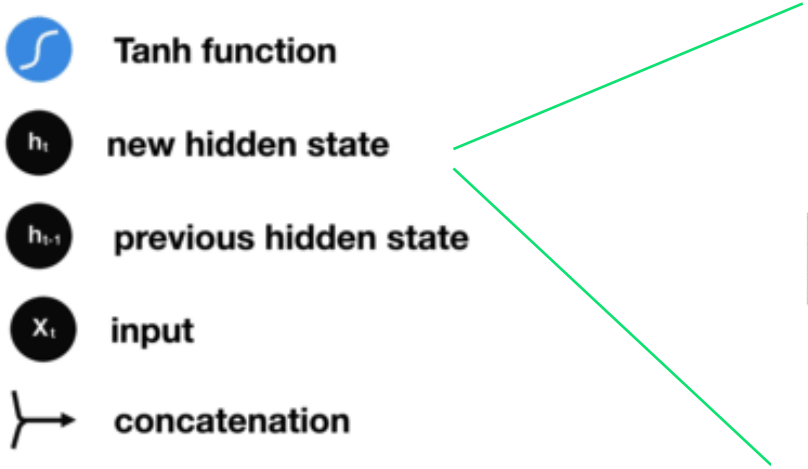


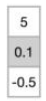
The Architecture behind RNNs

Recurrent Neural Networks

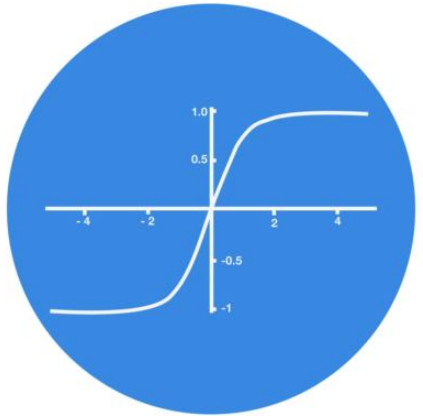
For each word:







Tanh



A tanh function ensures that the values stay between -1 and 1, thus regulating the output of the neural network, i.e., allowing the values to stay between the boundaries -1 and 1. Otherwise, some values can explode and become astronomical, causing other values to seem insignificant.

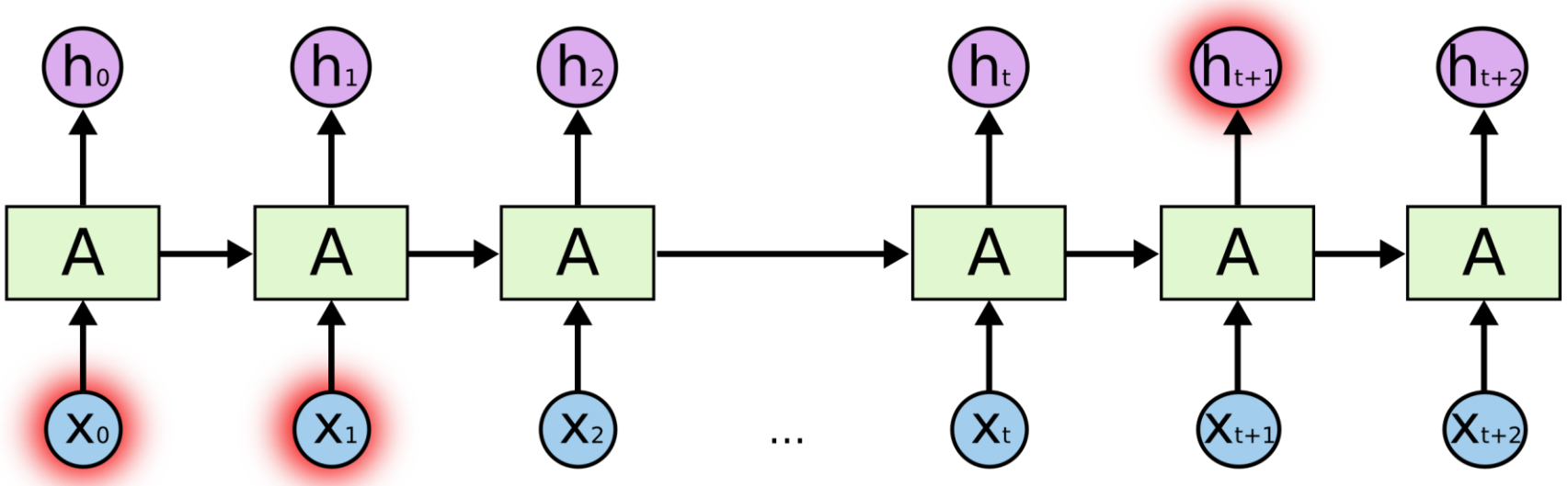
Problems with RNNs

- **Long Dependencies:**

The **final hidden state reflects more information about the end of the sentence** than its beginning.

Problems with RNNs

- **Long Dependencies**



Problems with RNNs

- **Long Dependencies**

Consider trying to **predict the last word in the text**

"I grew up in France (…) I speak fluent French."

Recent information suggests that the next word is probably the name of a language, but if we want to narrow down which language, **we need the context of France, from further back**.

Problems with RNNs

- **Vanishing Gradients**

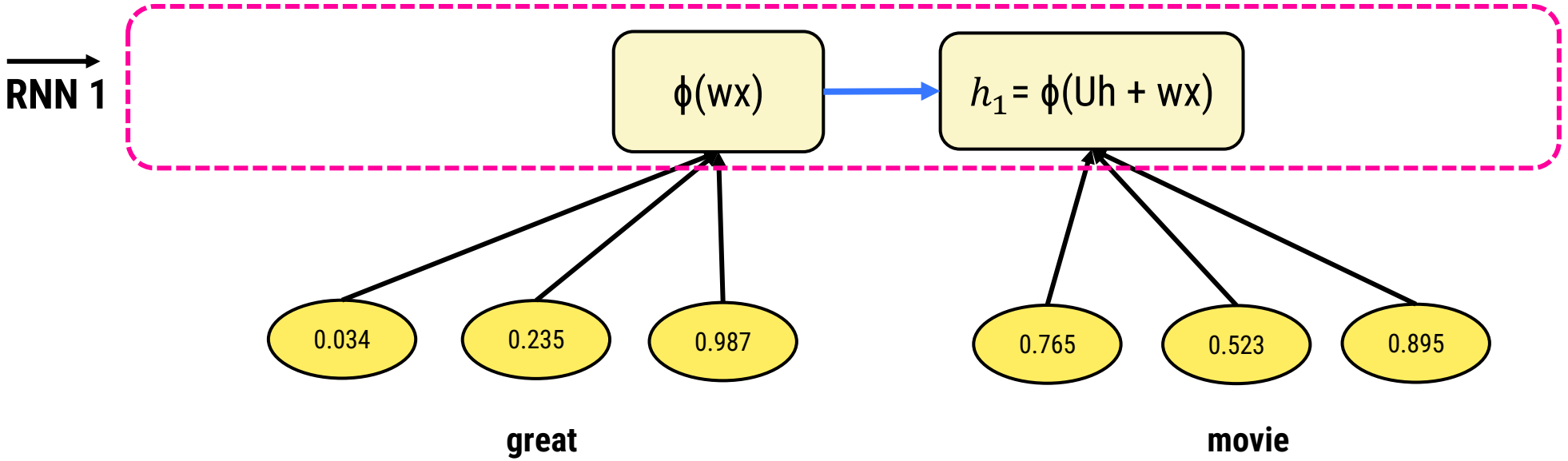
During the backward pass of training, the hidden layers are subject to repeated multiplications, as determined by the length of the sequence.

A frequent result of this process is that the **gradients are eventually driven to zero**, which means that the **initial weights cannot be updated properly!**

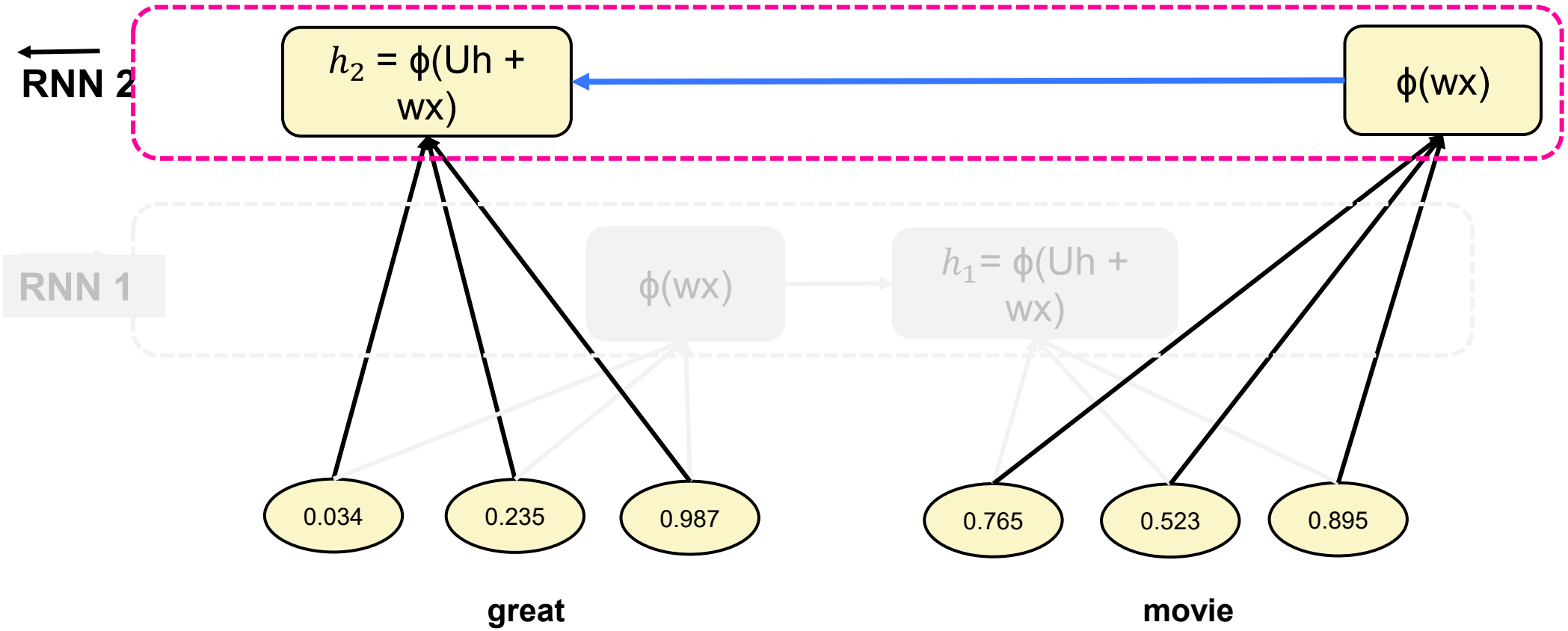
Problems with RNNs

A possible mitigation strategy for Long Dependecies: Make the model **Bidirectional**!

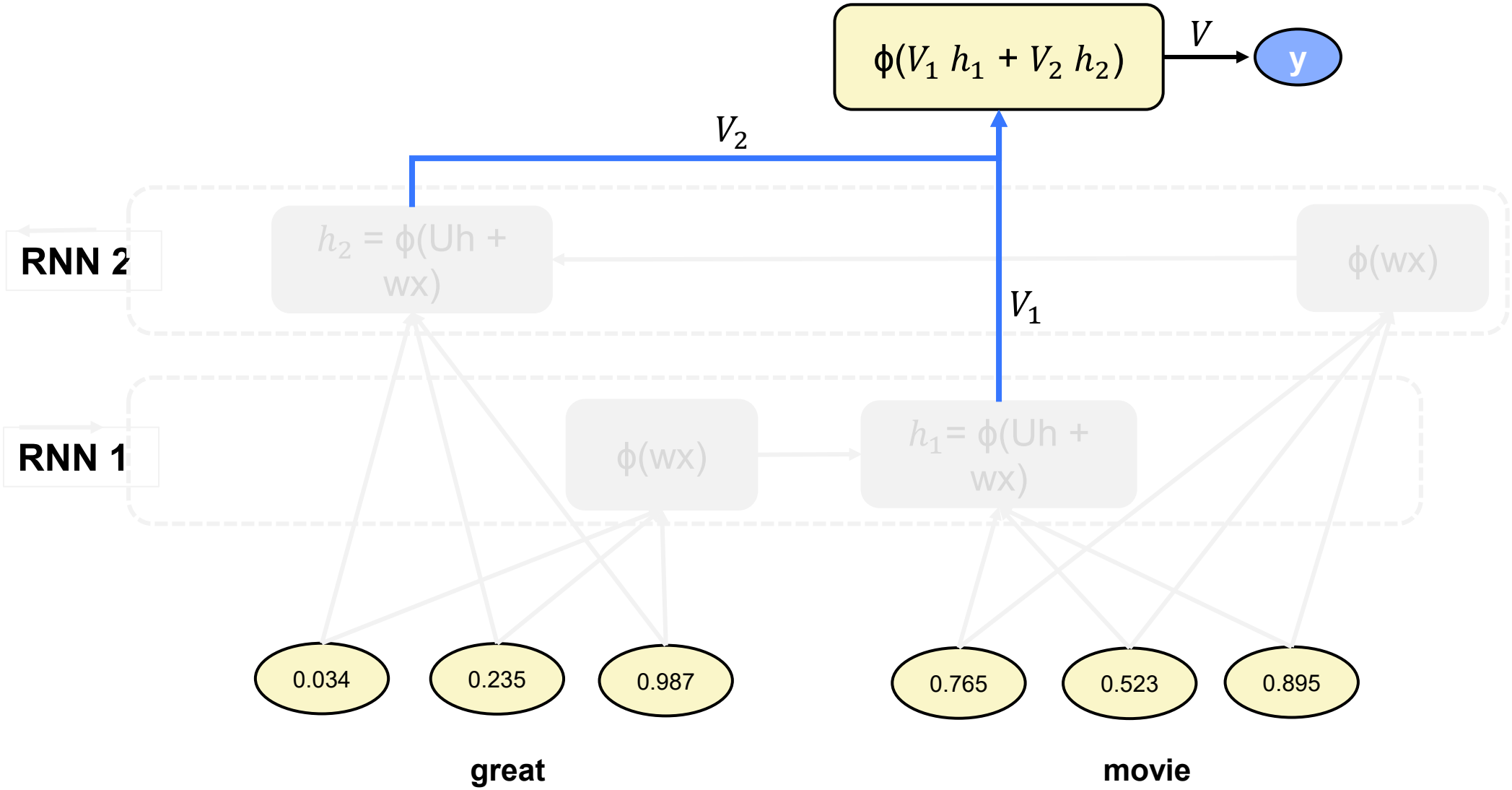
Left to Right:



Right to Left:



Bi-directionality in Recurrent Neural Networks



03.
Long Short-Term Memory Neural Networks

Problems with RNNs

Mitigation Strategy: Make the model Bidirectional.

Possible Solution: **Long Short Term Memory (LSTM) Neural Networks**

LSTMs are explicitly designed to **avoid the long-term dependency** problem.

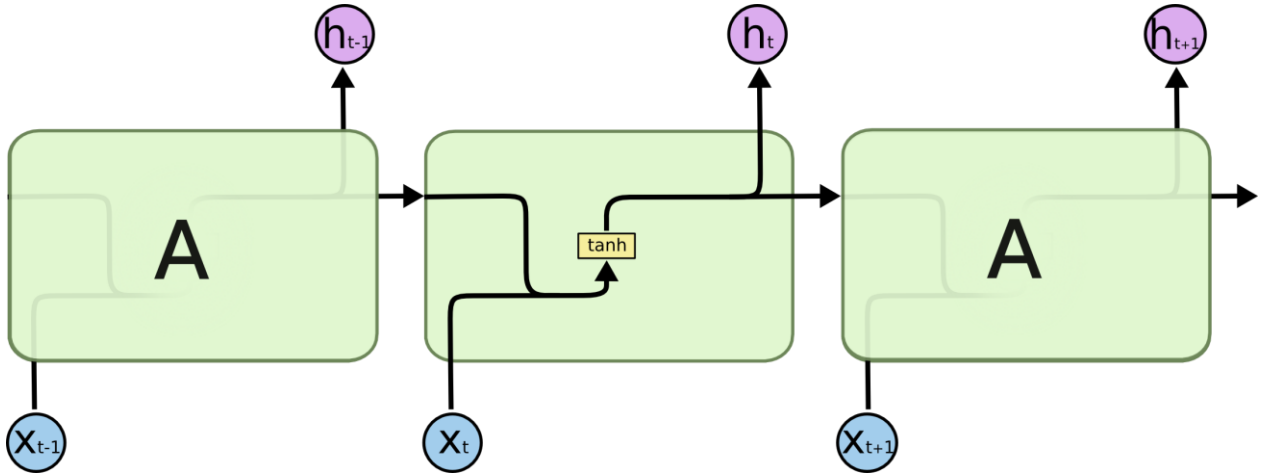
LSTMs divide the context management problem into **two subproblems**:

1. **removing information** no longer needed from the context
2. **adding information** likely to be needed for later decision making.

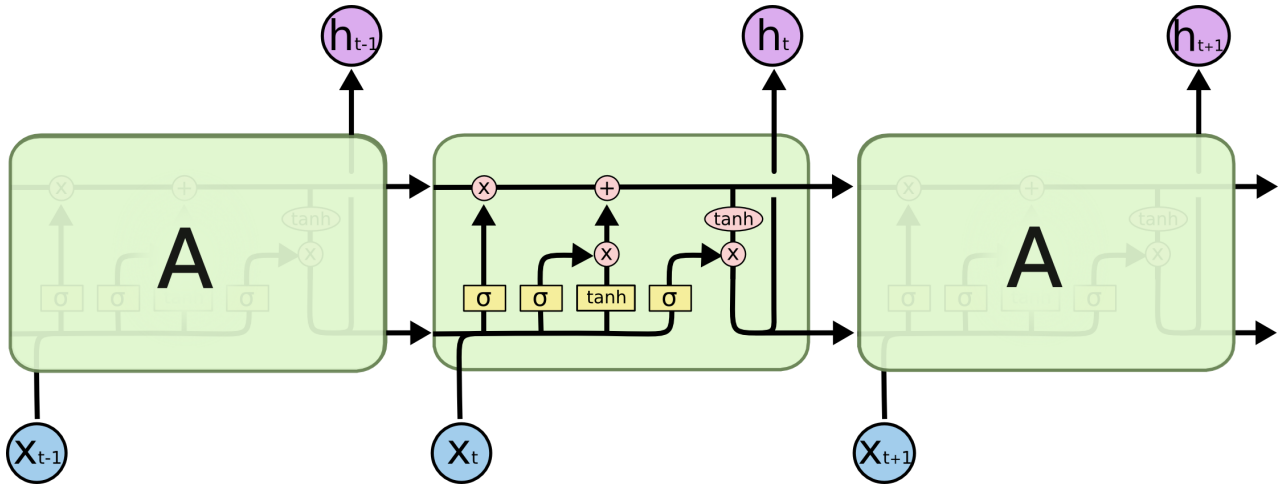
The Architecture behind LSTMs

Long Short-Term Memory Neural Networks

RNN



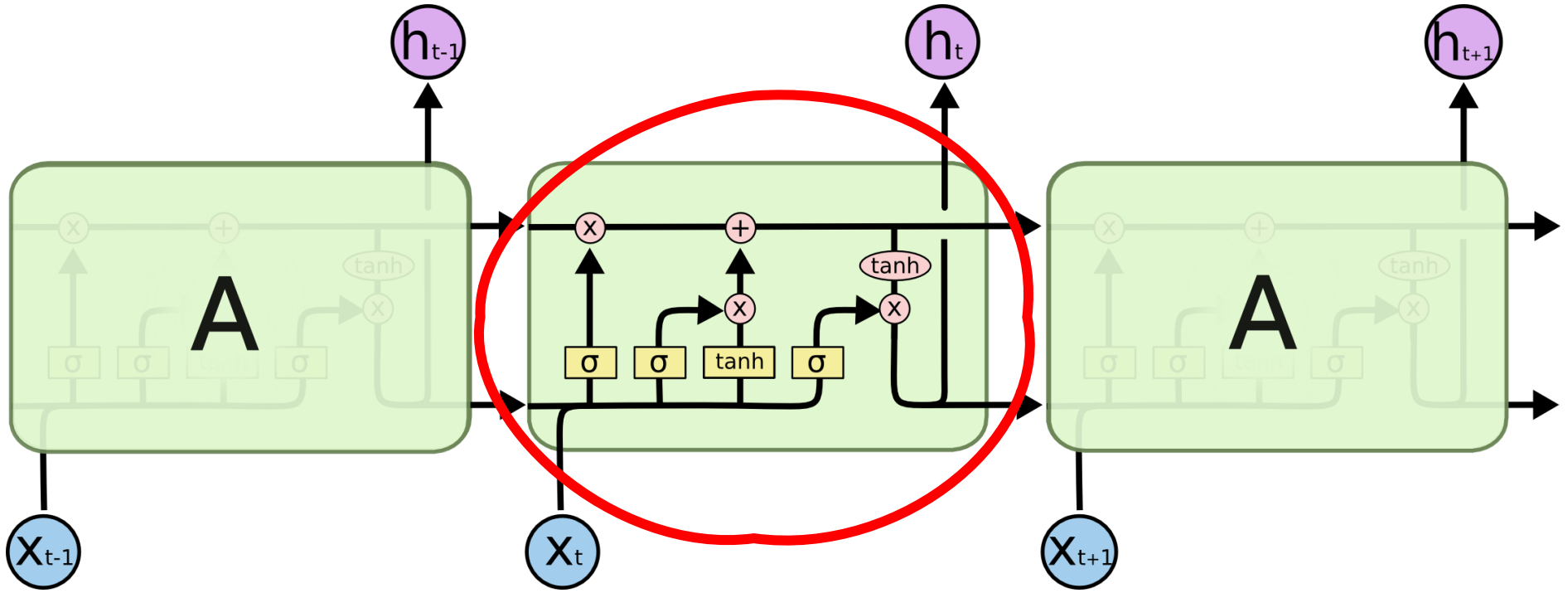
LSTM



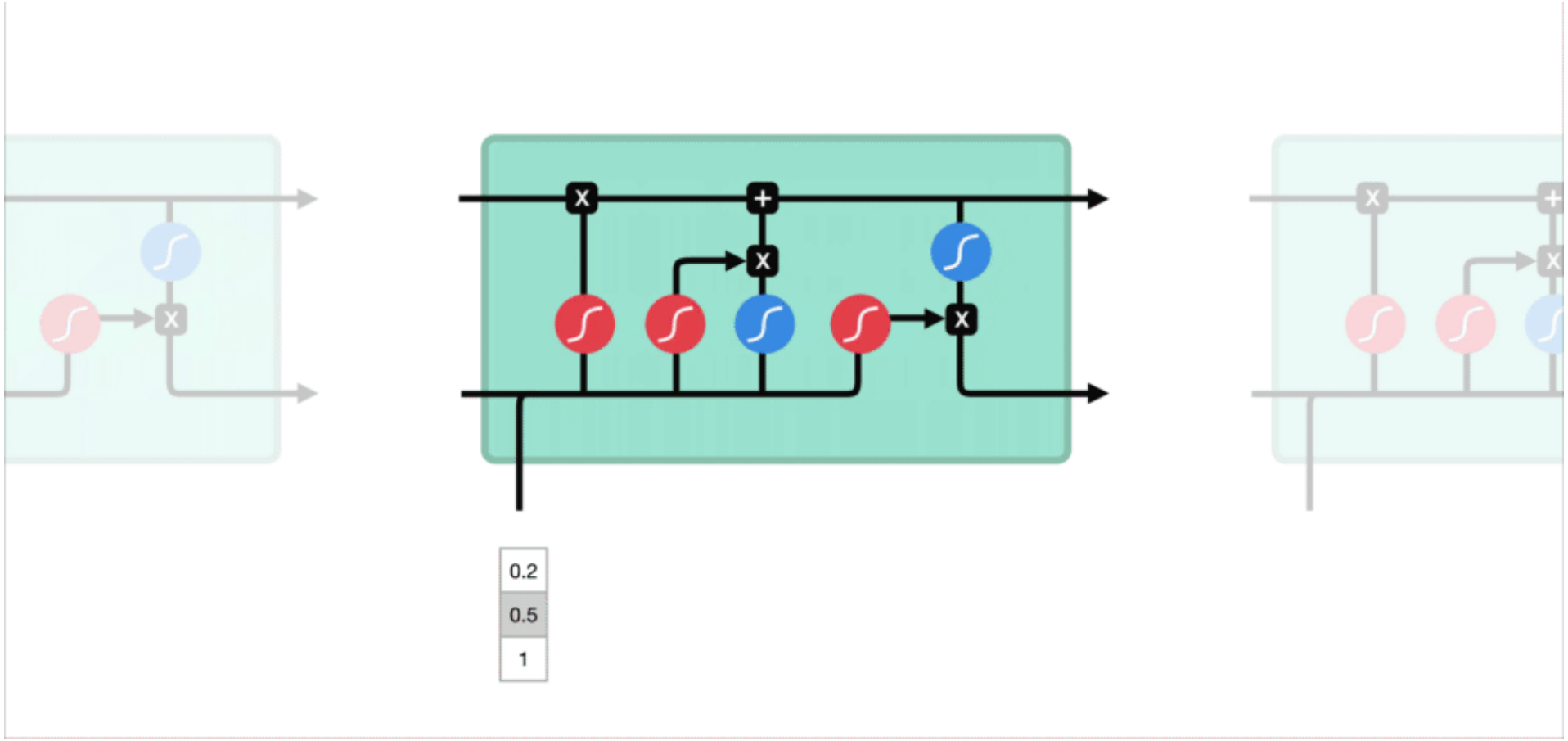
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

The Architecture behind LSTMs

Long Short-Term Memory Neural Networks



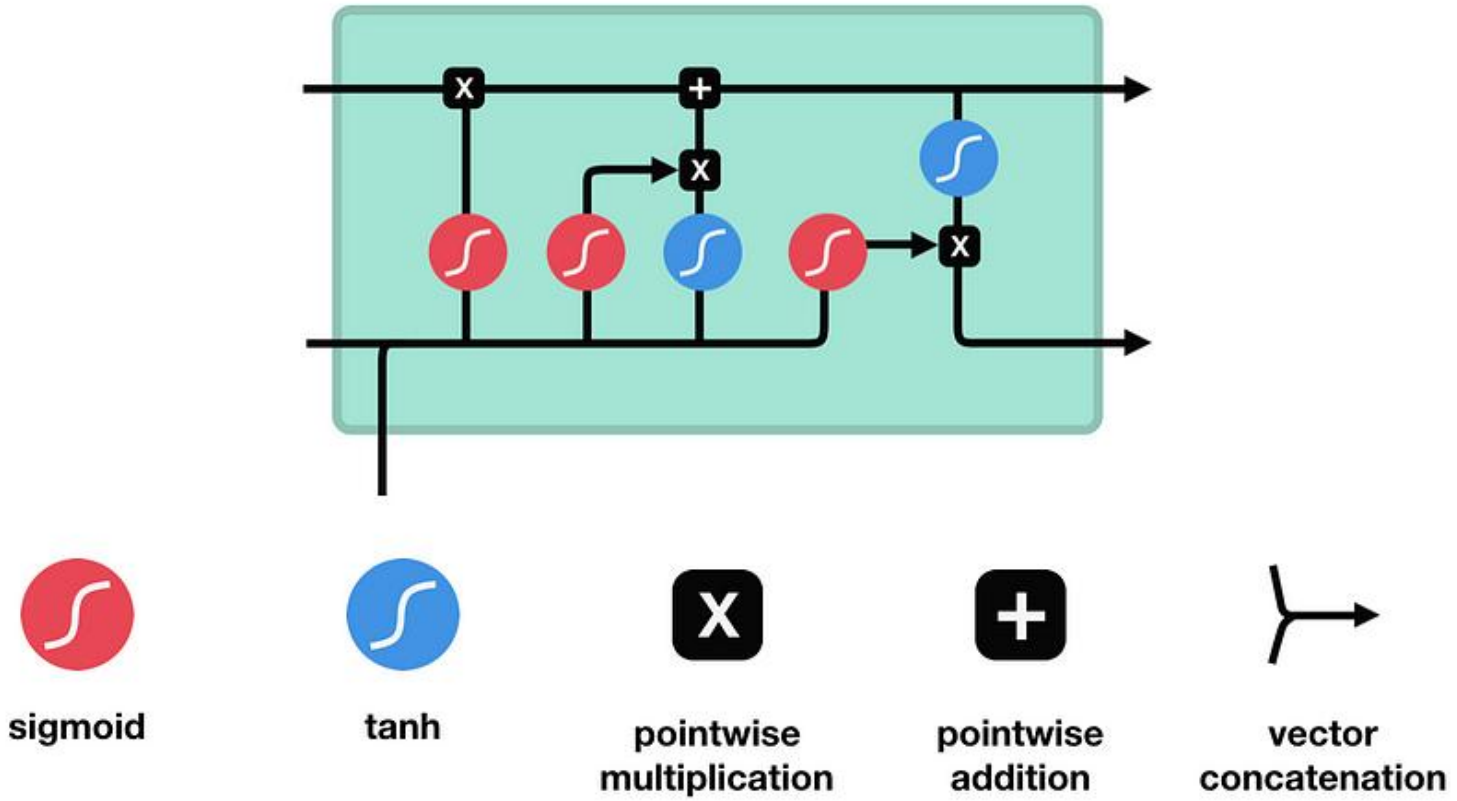
For each sentence, each word embedding is processed through the network:



https://www.youtube.com/watch?v=8HyCNIVRbSU

The Architecture behind LSTMs

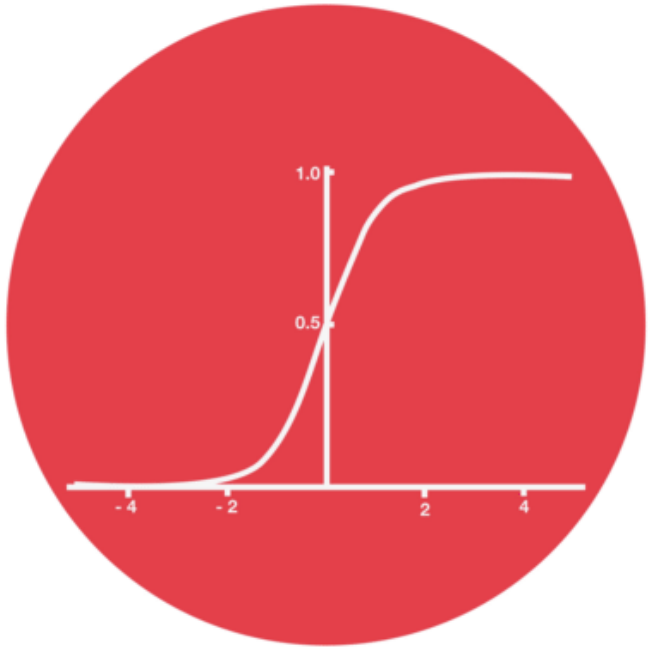
Long Short-Term Memory Neural Networks



https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21

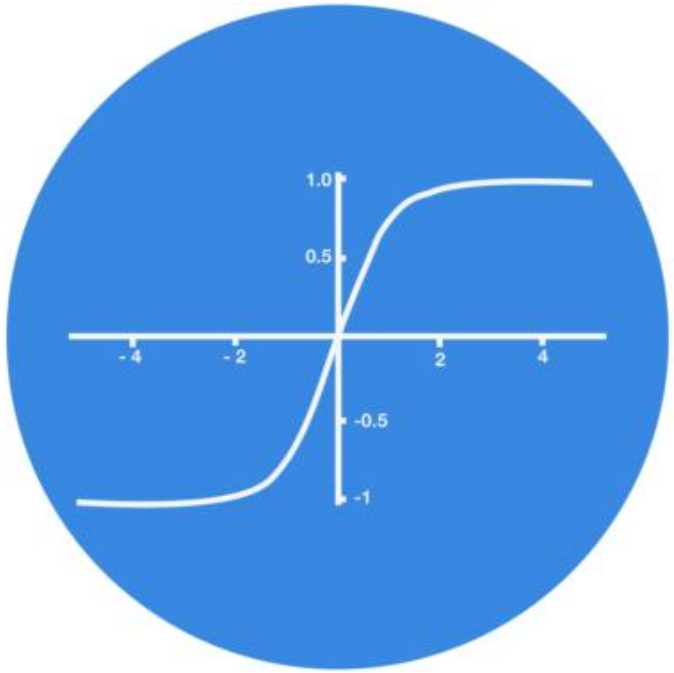
Sigmoid (Logistic) Activation

5
0.1
-0.5



Tanh Activation

5
0.1
-0.5



LSTMs are explicitly designed to **avoid the long-term dependency problem**.

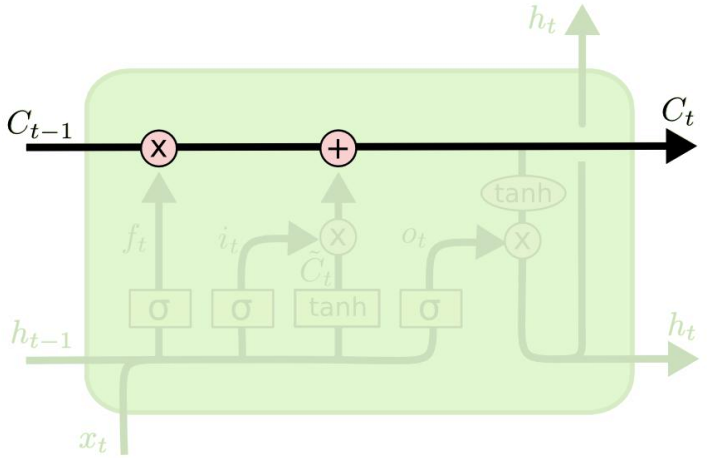
To accomplish this, LSTMs make use of:

- Context layer (the **cell state**)
- A specialized neural units that work as **gates to control the flow of information** into and out of the units that comprise the network layers.

Cell State

The key to LSTMs is the cell state, the **horizontal line running through the top** of the diagram.

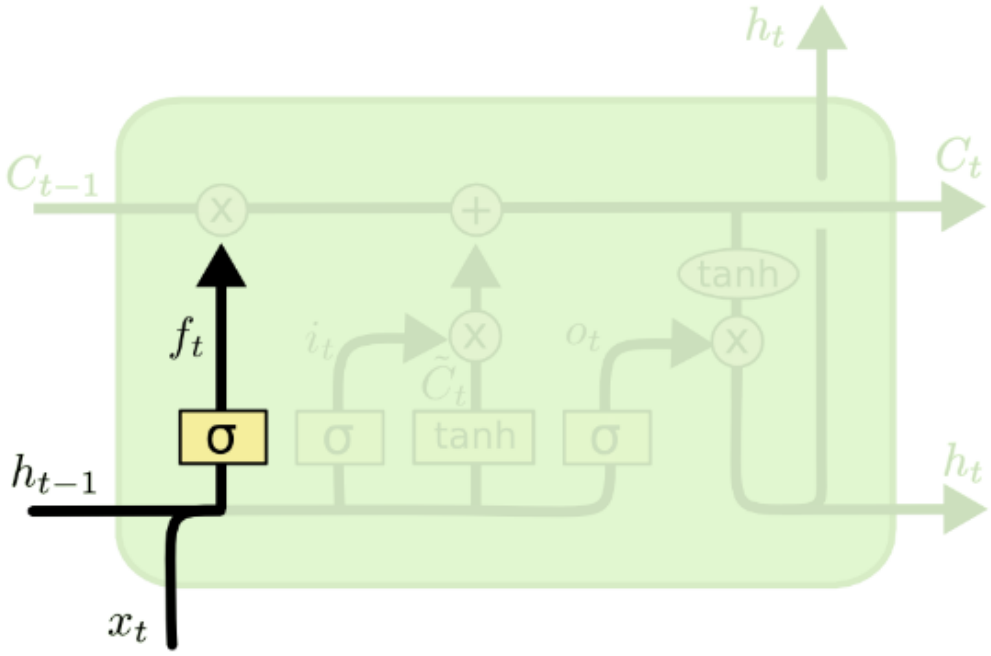
The cell state acts as a transport highway that **transfers relative information all the way down the sequence chain**. You can think of it as the "memory" of the network.



Gates in LSTM

1. **Forget Gate**
2. Input Gate
3. Output Gate

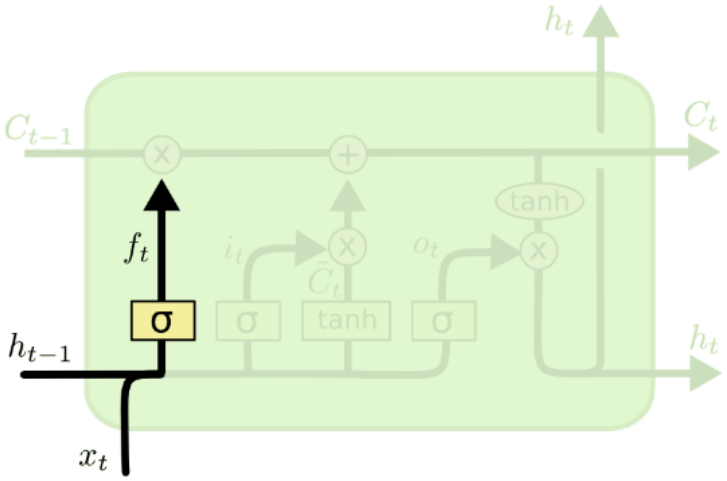
1. Forget Gate



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

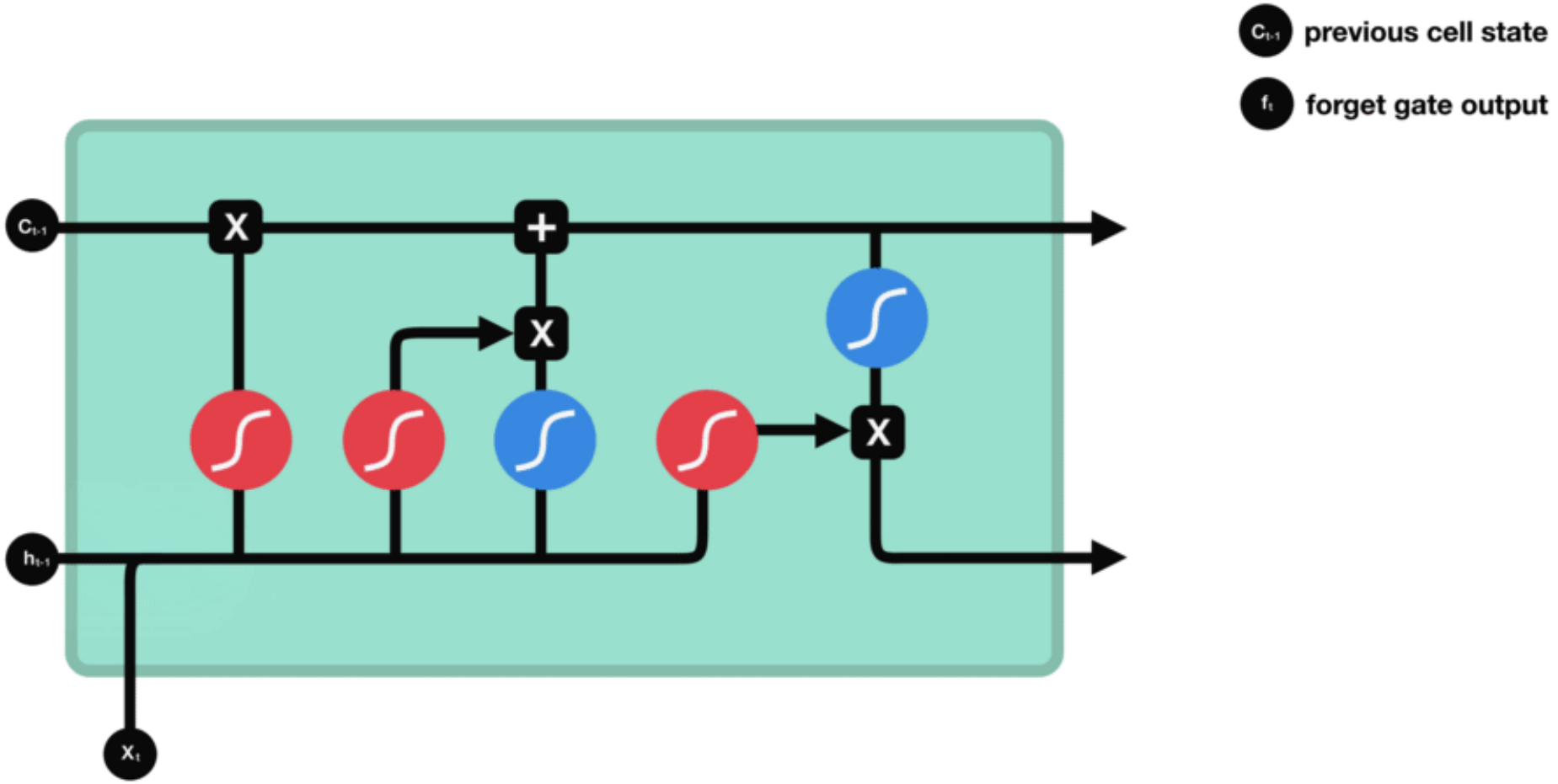
1. Forget Gate

This gate decides **what information should be thrown away** or kept in the cell state. Information from the previous hidden state and information from the current input is passed through the sigmoid function. Values come out between 0 and 1. The closer to 0 means to forget, and the closer to 1 means to keep.



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

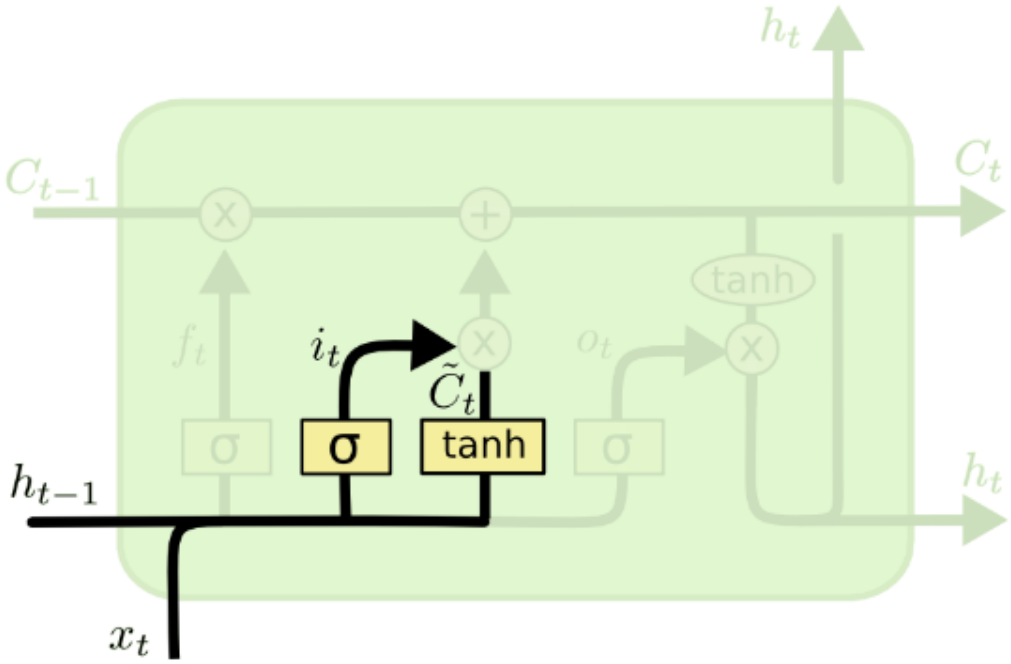
1. Forget Gate



Gates in LSTM:

1. Forget Gate
2. **Input Gate**
3. Output Gate

2. Input Gate



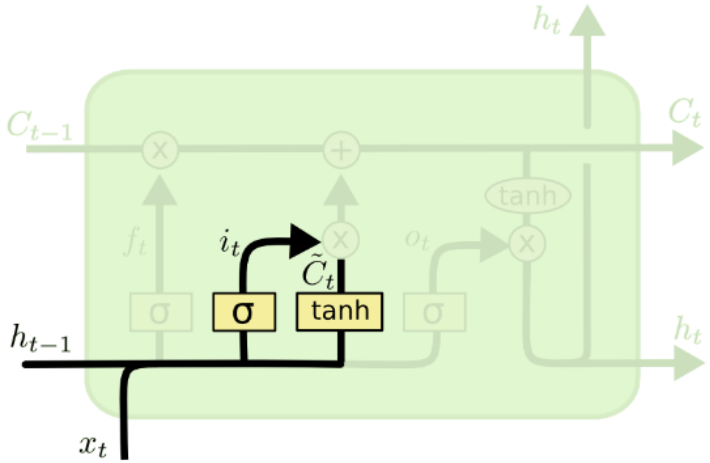
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

2. Input Gate

This gate decides what information should be used to **update** the cell state.

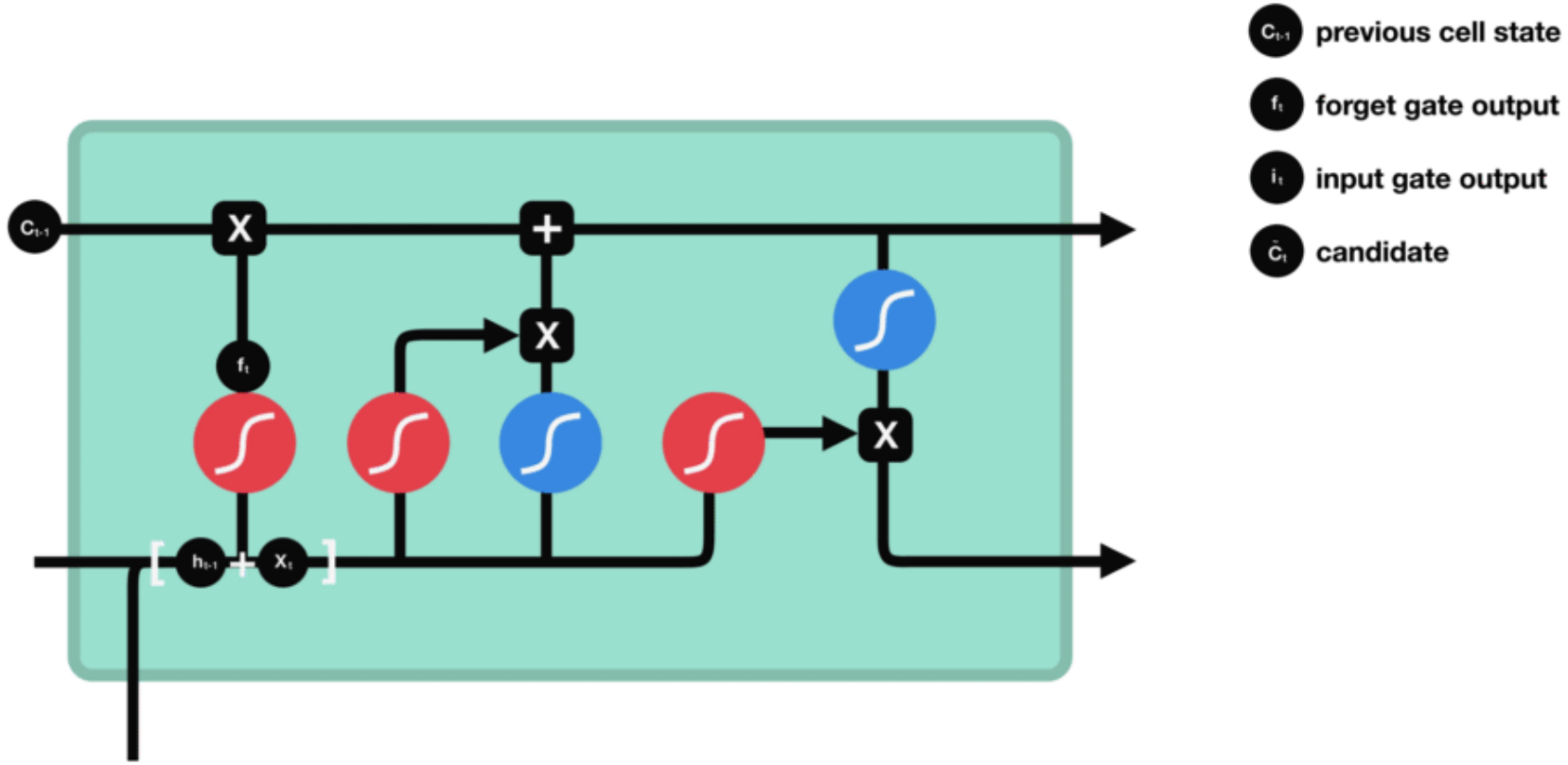
1. First, we **pass the previous hidden state and current input into a sigmoid function**. That decides which values will be updated by transforming the values to be between 0 and 1. 0 means not important, and 1 means important.
2. You also **pass the hidden state and current input into the tanh function** to squash values between -1 and 1 to help regulate the network.
3. Then you **multiply the tanh output with the sigmoid output**. The sigmoid output will decide which information is important to keep from the tanh output.



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

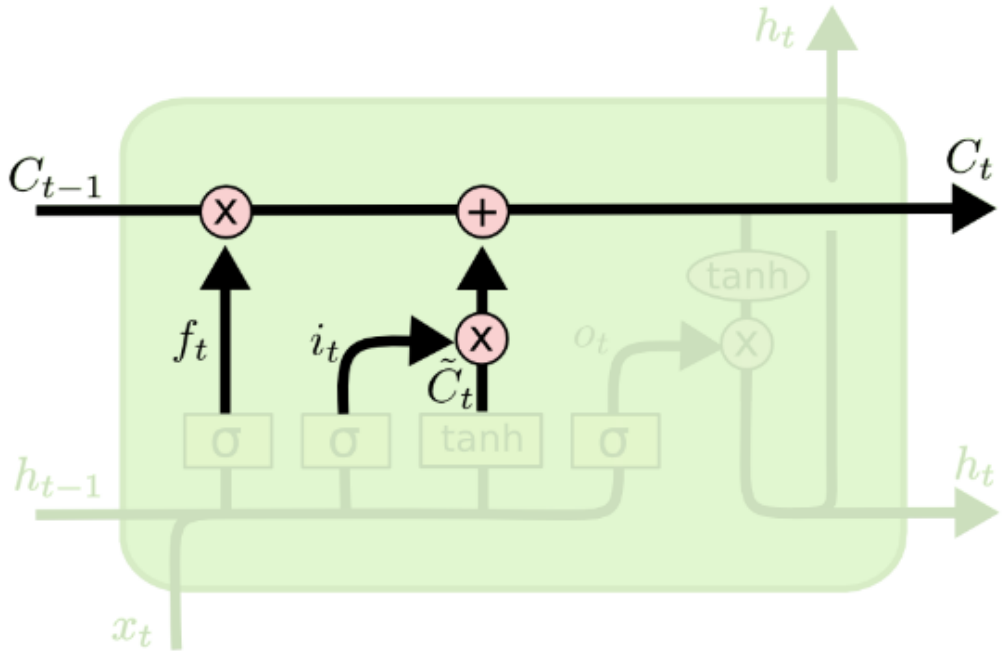
2. Input Gate


previous cell state
forget gate output
input gate output
candidate

Gates in LSTM:

1. Forget Gate
2. Input Gate
    2.5. Calculate Cell State
3. Output Gate

2.5. Calculate the Cell State

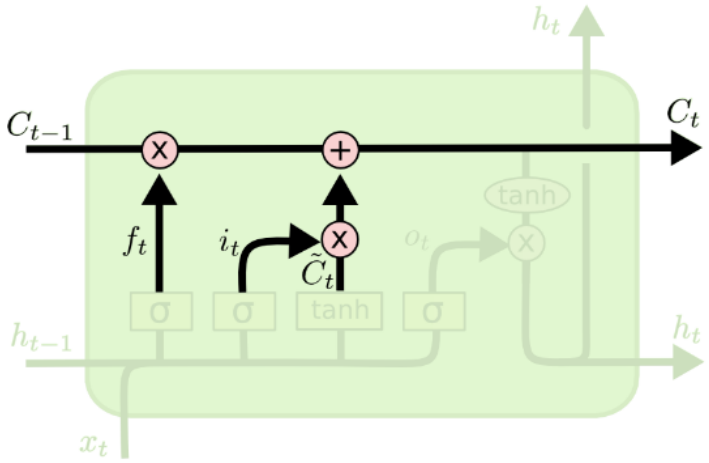


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

2.5. Calculate the Cell State

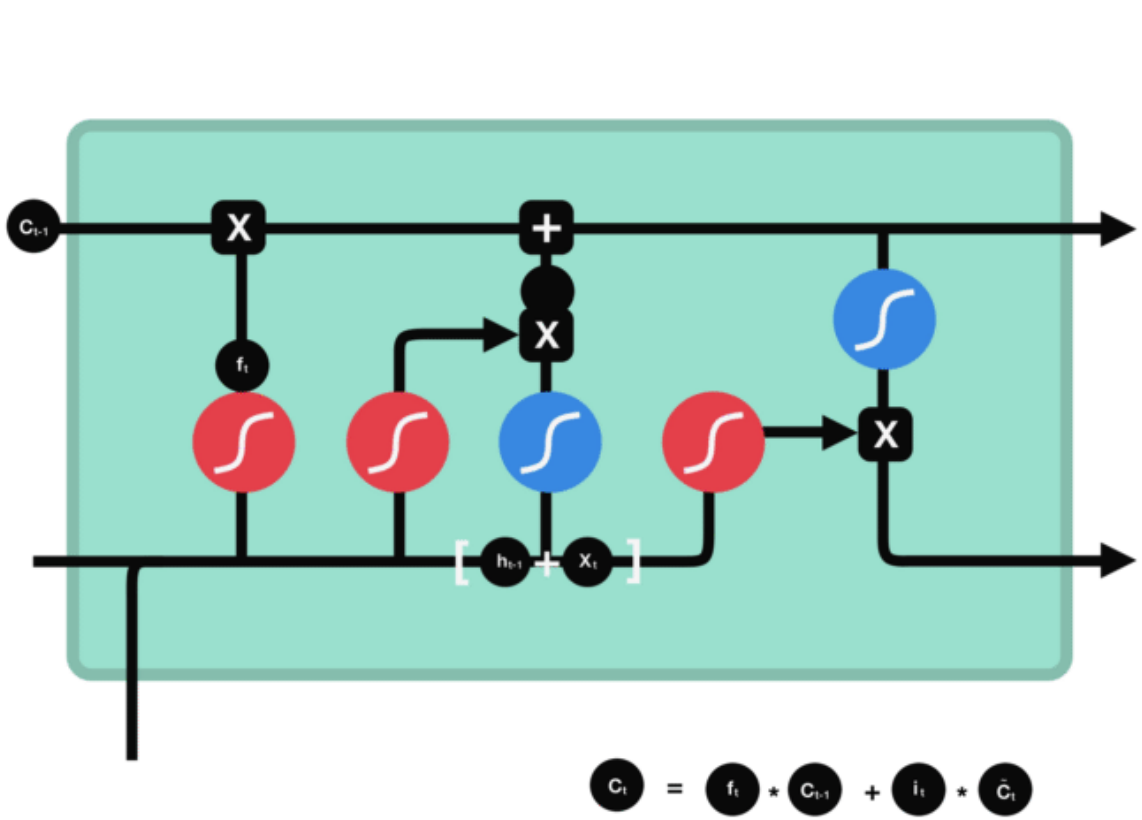
1. The **cell state gets pointwise multiplied by the forget vector**. This has a possibility of dropping values in the cell state if it gets multiplied by values near 0.

2. Then we **take the output from the input gate and do a pointwise addition** which updates the cell state to new values that the neural network finds relevant.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

2.5. Calculate the Cell State



C_{t-1} previous cell state

f_t forget gate output

i_t input gate output

$\tilde{C}_t$ candidate

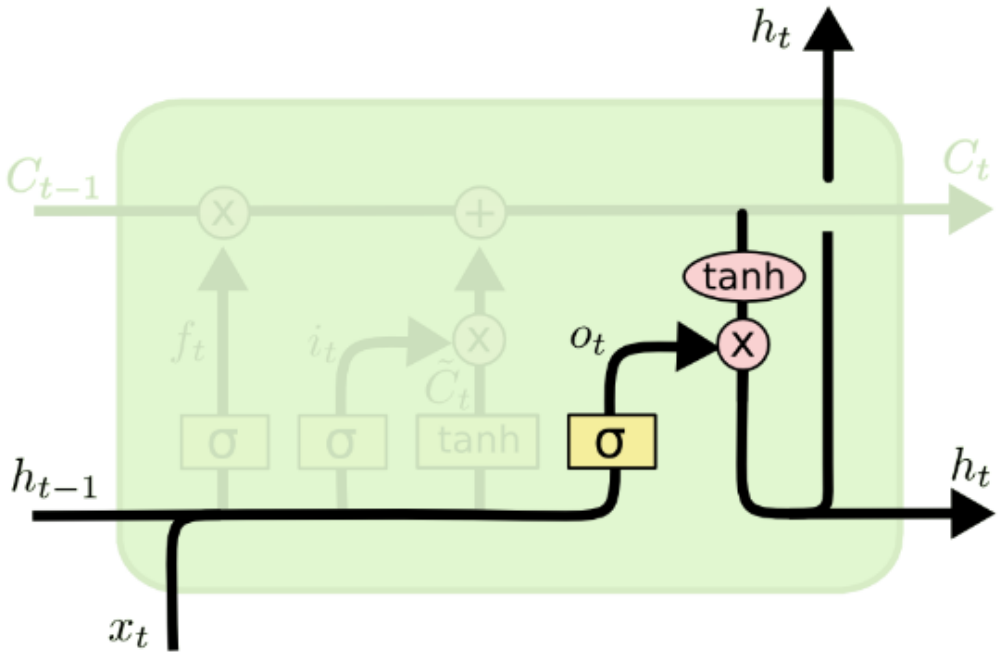
C_t new cell state

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Gates in LSTM:

1. Forget Gate
2. Input Gate
    2.5. Calculate Cell State
3. **Output Gate**

3. Output Gate



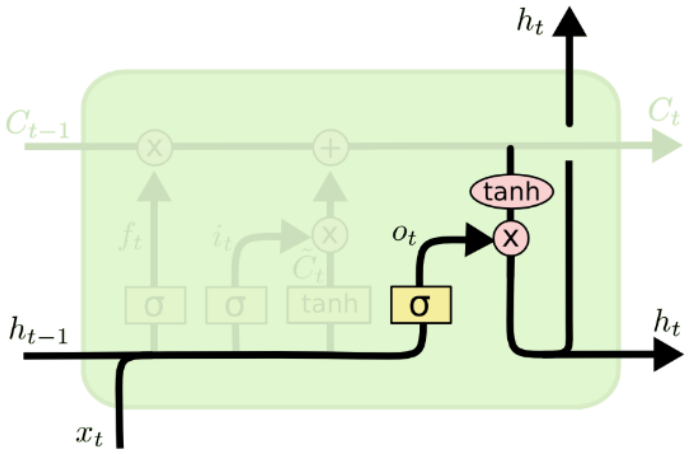
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

3. Output Gate

This gate decides what is the next hidden state or, if we are at the last step of the network, what is our prediction.

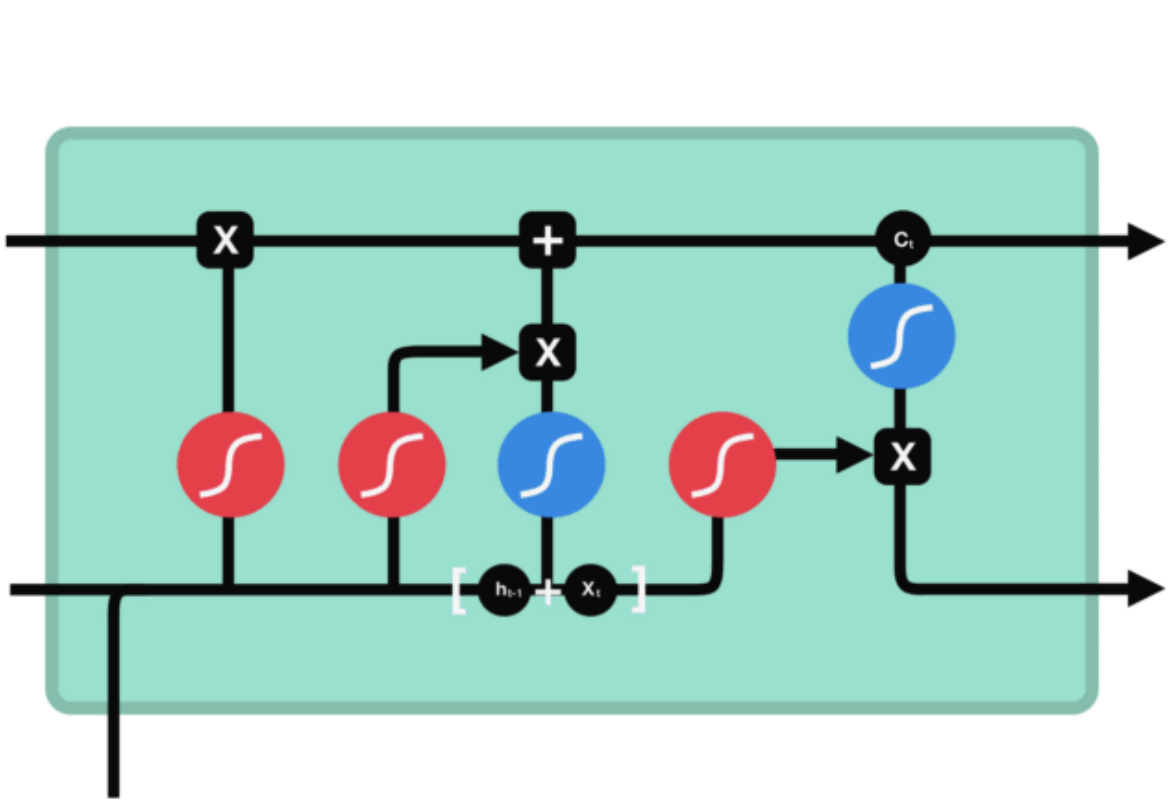
1. First, we **pass the previous hidden state and the current input into a sigmoid function**.
2. Then we **pass the newly modified cell state to the tanh function**.
3. We **multiply the tanh output with the sigmoid output** to decide what information the hidden state should carry. The output is the hidden state. The new cell state and the new hidden is then **carried over to the next time step** or passed to another activation layer to **give the prediction output**.



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

3. Output Gate



C_{t-1} previous cell state

f_t forget gate output

i_t input gate output

$\tilde{C}_t$ candidate

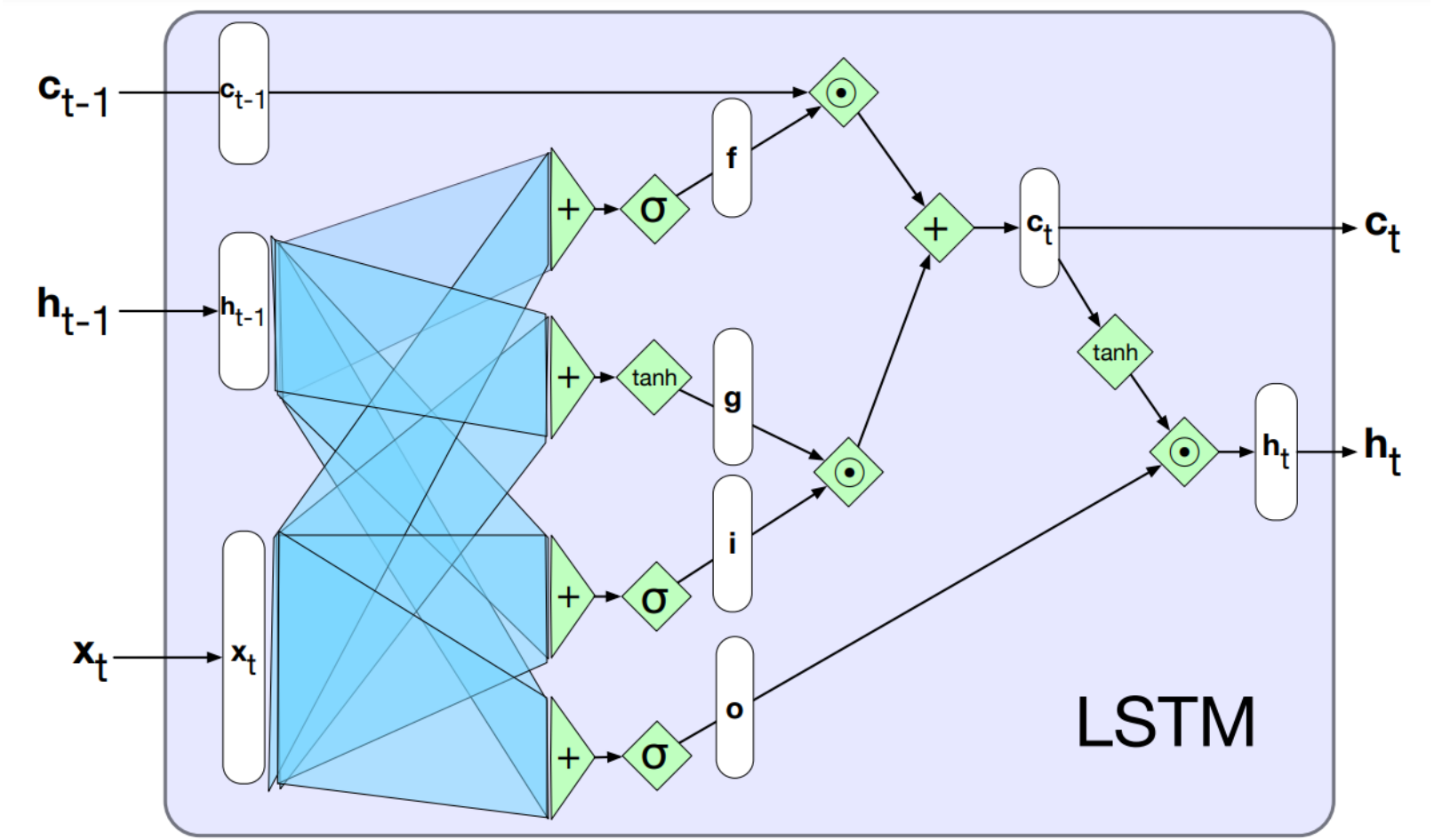
C_t new cell state

o_t output gate output

h_t hidden state

The Architecture behind LSTMs

Long Short-Term Memory Neural Networks



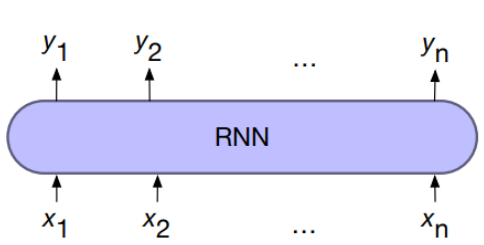
https://web.stanford.edu/~jurafsky/slp3/9.pdf

04. Sequence-to-Sequence Models (Seq2Seq)

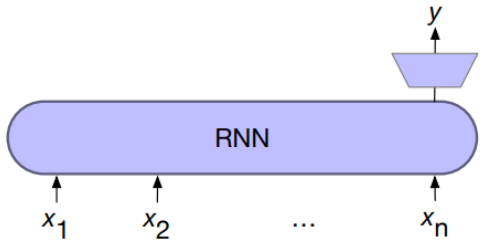
To process sequential input, we can use some well know sequential models, like **Recurrent Neural Networks (RNN)** and **Long Short Term Memory Neural Networks (LSTM)**

Common Tasks

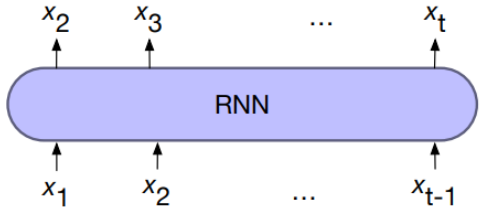
- Next word prediction
- Sequence Labeling
- Sequence Classification
- **Machine Translation**
- **Chatbots**



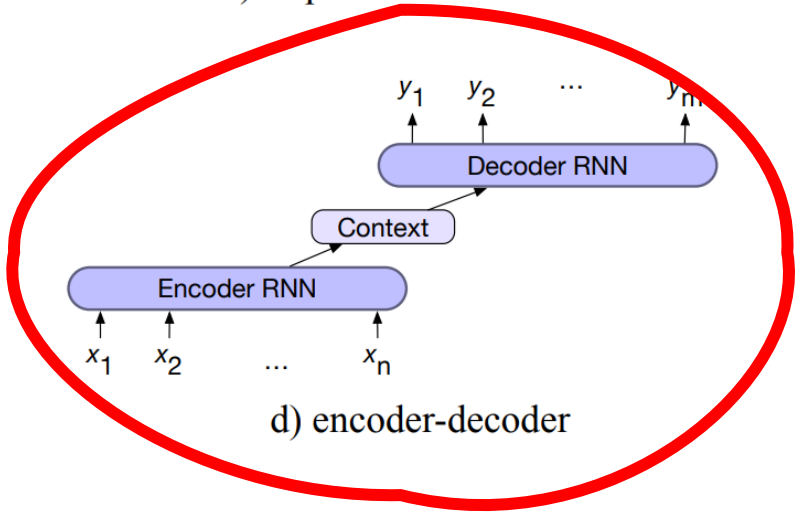
a) sequence labeling



b) sequence classification



c) language modeling



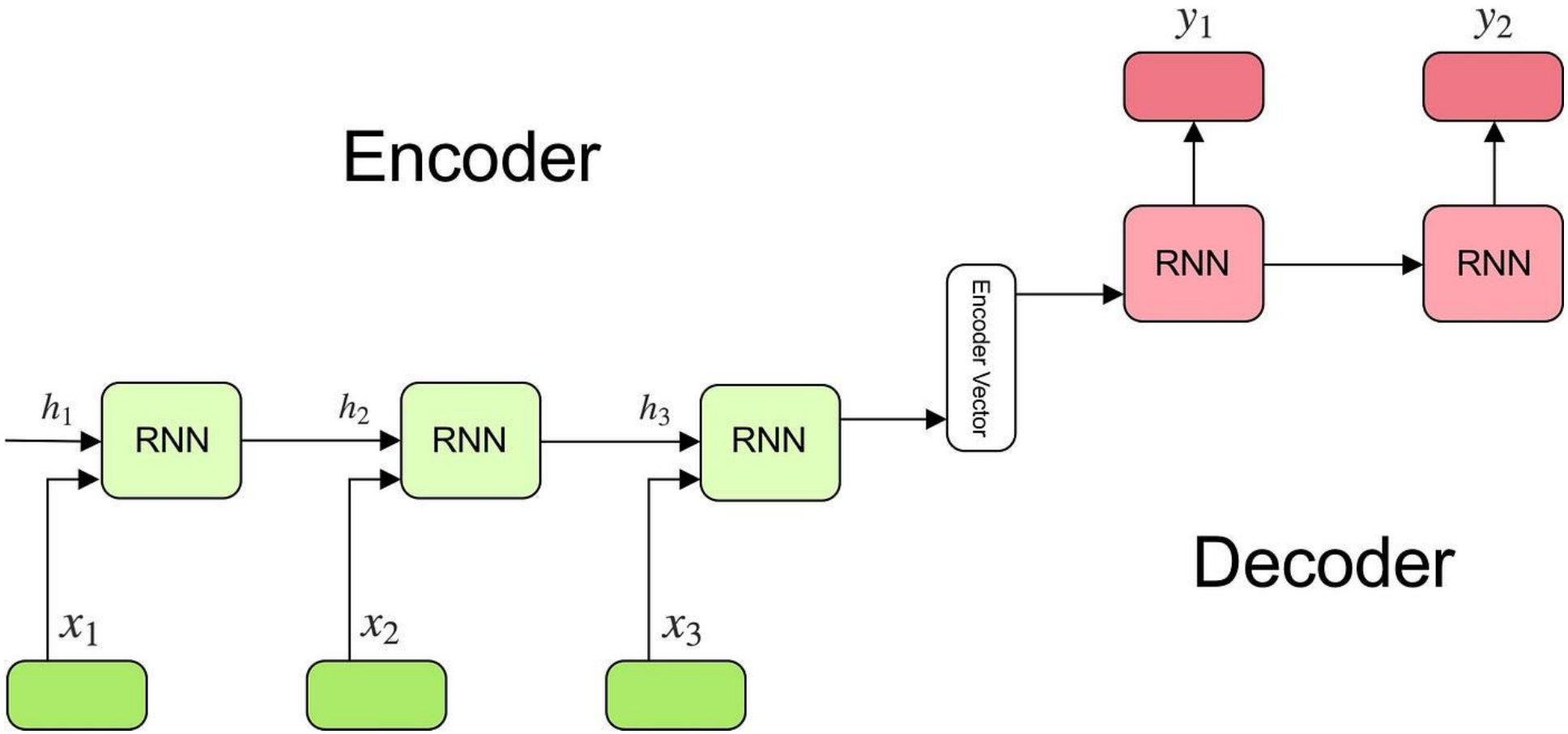
d) encoder-decoder

Sequence-to-sequence (seq2seq) are a type of neural network architecture that can be used for tasks such as machine translation, text summarization, and speech recognition.

The **input and output are both sequences of variable length**, and the model maps one sequence to another. For example, in machine translation, the input sequence is a sentence in one language, and the output sequence is the translation of that sentence in another language.

The Architecture behind Seq2Seq

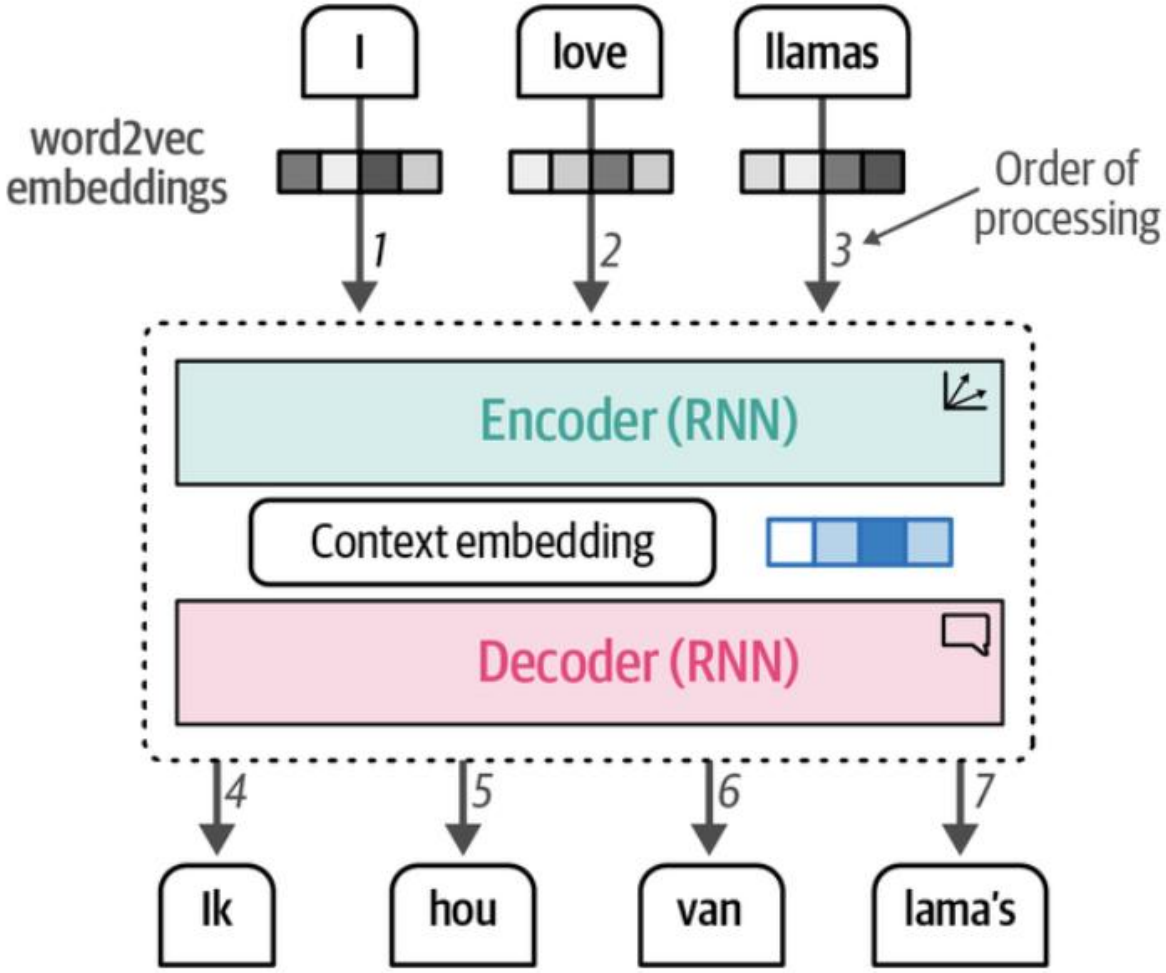
Sequence-to-Sequence Models



https://link.springer.com/chapter/10.1007/978-981-16-6723-7_23

The Architecture behind Seq2Seq

Sequence-to-Sequence Models



The input sequence (e.g. a sentence) is fed into an **encoder neural network**, which produces a **fixed-length representation of the input sequence** (similar to a sentence embedding).

The encoder typically uses a **recurrent neural network (RNN)**, such as a long short-term memory (LSTM) network, to process the input sequence.

At the end of the input sequence, **the final hidden state of the encoder is used as the initial hidden state of a decoder network**.

$$h_t = f(W^{(hh)} h_{t-1} + W^{(hx)} x_t)$$

The **decoder network** is also typically some RNN. At each time step, the **decoder takes the previous element** of the output sequence as input, along with the previous hidden state of the decoder.

The **input to the decoder at time t is typically the previously generated element** of the output sequence, y_{t-1} which is typically represented as a one-hot vector.

Each **recurrent unit accepts a hidden state from the previous unit** and produces an output as well as its own hidden state.

$$h_t = f(W^{(hh)} h_{t-1})$$

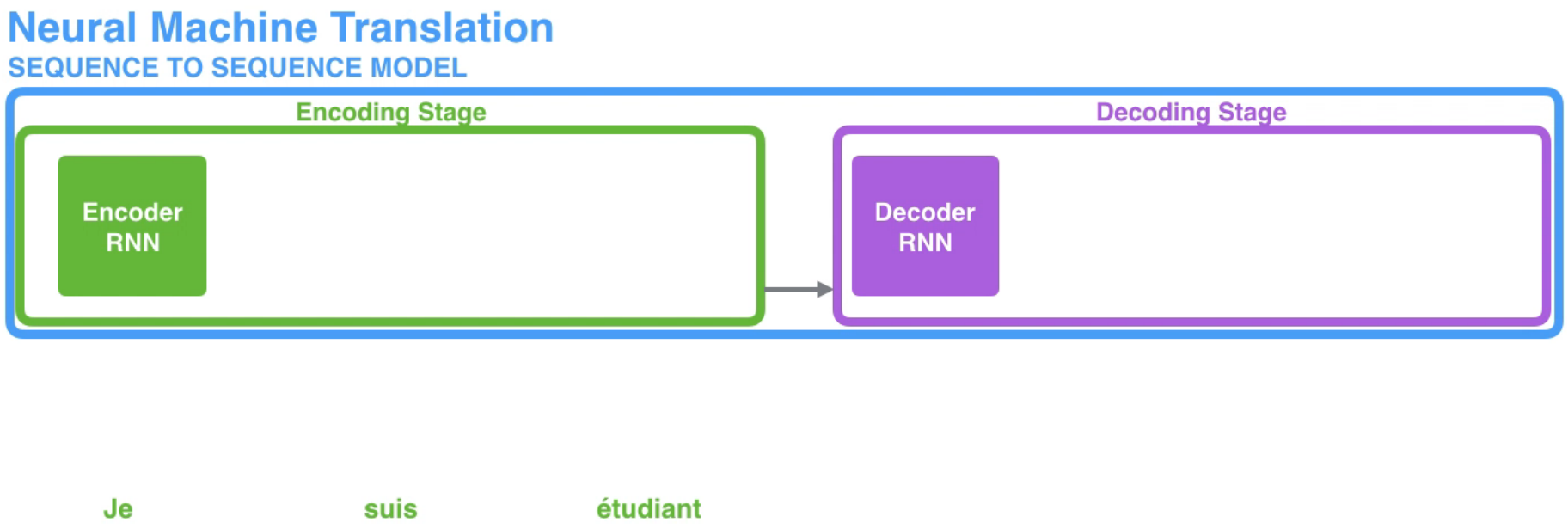
The output is calculated using **the hidden state at the current time step together with the respective weight matrix**.

Softmax is used to **create a probability vector which will help us determine the final output** (e.g. word in the question-answering problem).

$$y_t = softmax(W^S h_t)$$

The Architecture behind Seq2Seq

Sequence-to-Sequence Models



https://jalammar.github.io/images/seq2seq_6.mp4

Example:

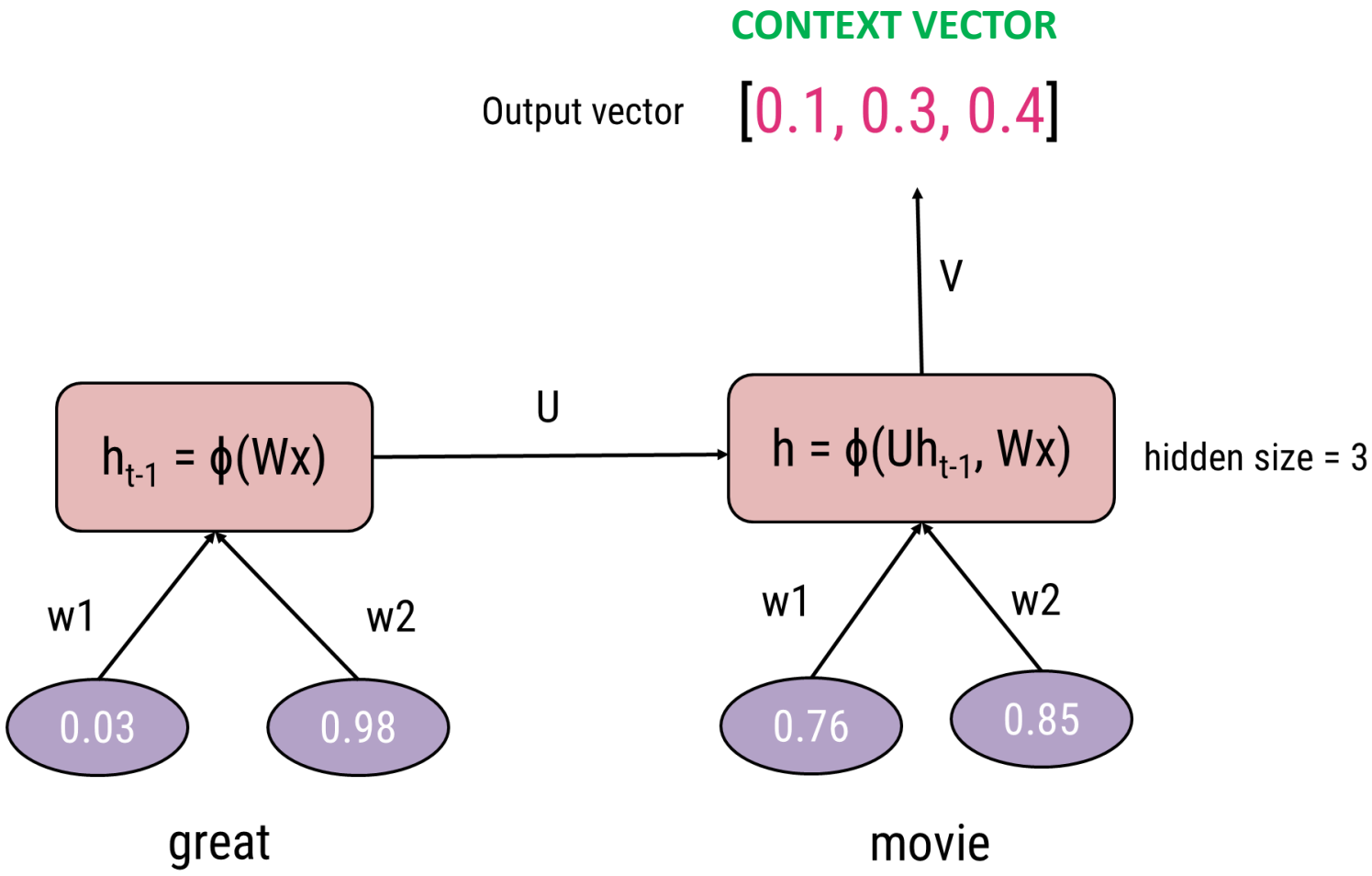
Sentence – “Great movie”

Translation – “Ótimo filme”

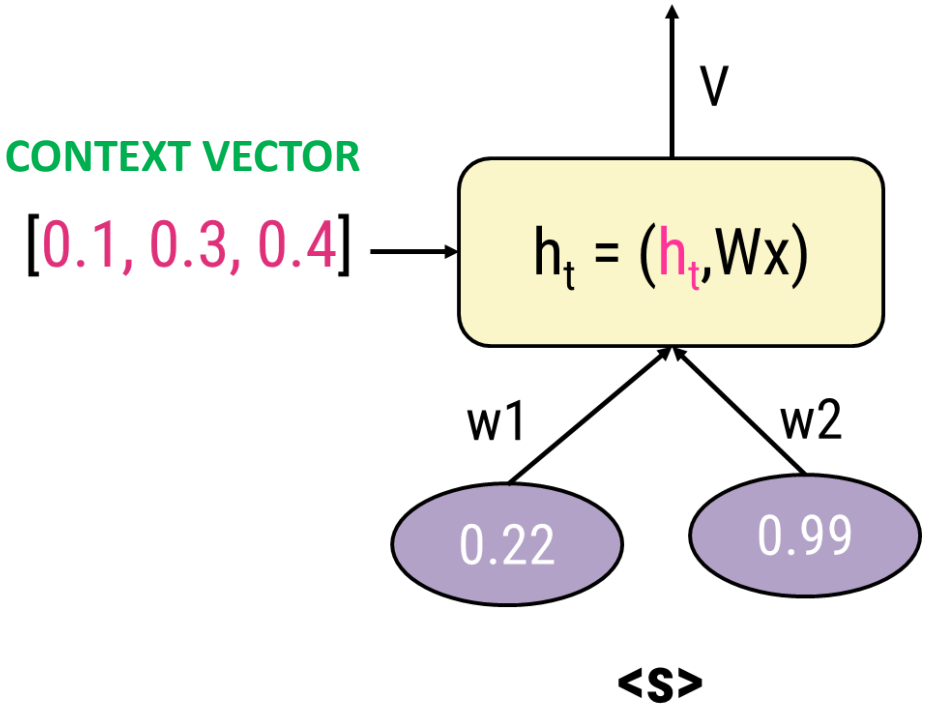
	Word Embeddings (size 2)	
	great	movie
Sentence	[0.03, 0.98]	[0.76, 0.85]

	Word Embeddings (size 2)			
	<s>	ótimo	filme	</s>
Translation	[0.22, 0.99]	[0.40, 0.23]	[0.05, 0.33]	[0.01, 0.76]

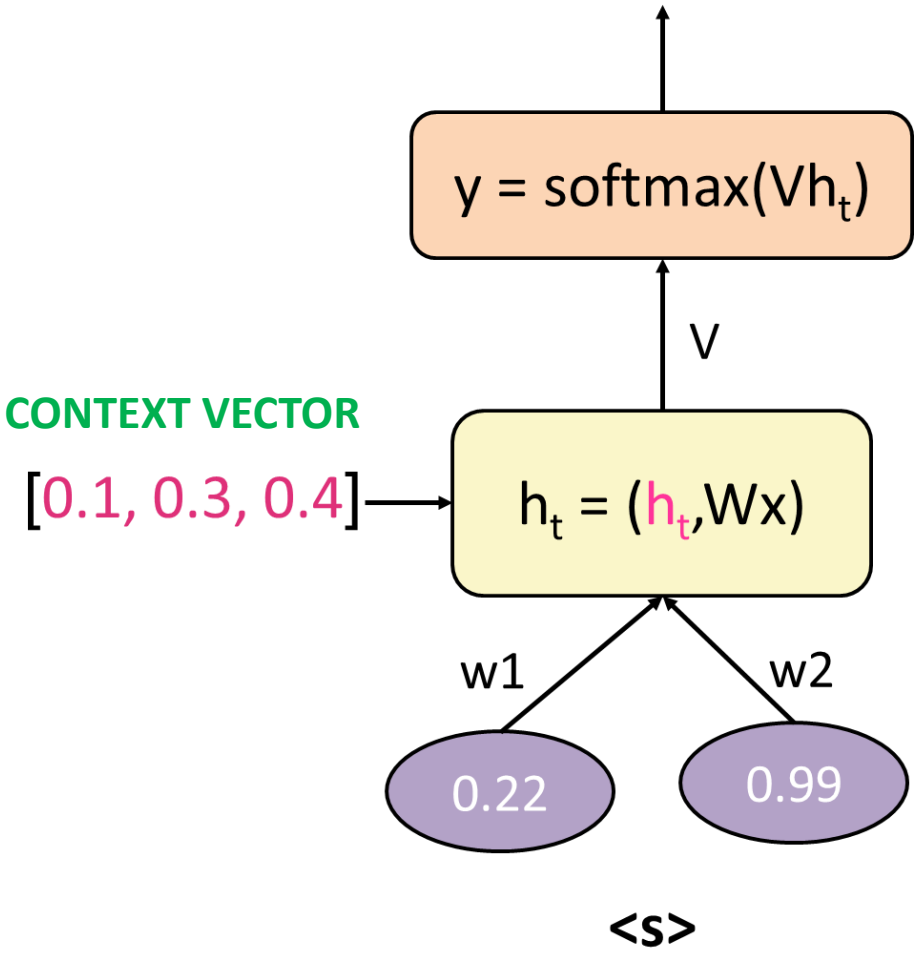
Encoder

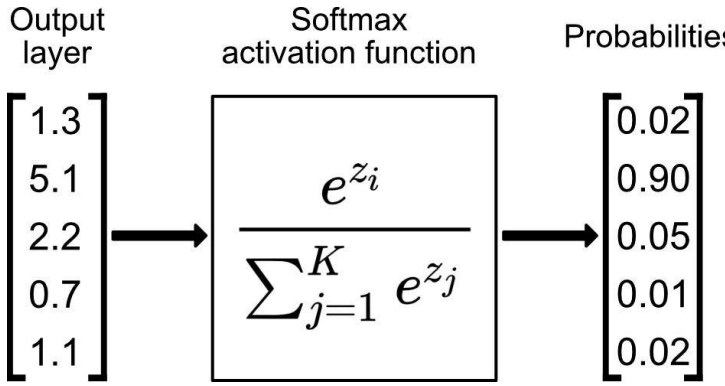

CONTEXT VECTOR
Output vector
[0.1, 0.3, 0.4]
V
U
$h_{t-1} = \phi(Wx)$
$h = \phi(Uh_{t-1}, Wx)$
hidden size = 3
w1
w2
w1
w2
0.03
0.98
0.76
0.85
great
movie

Decoder

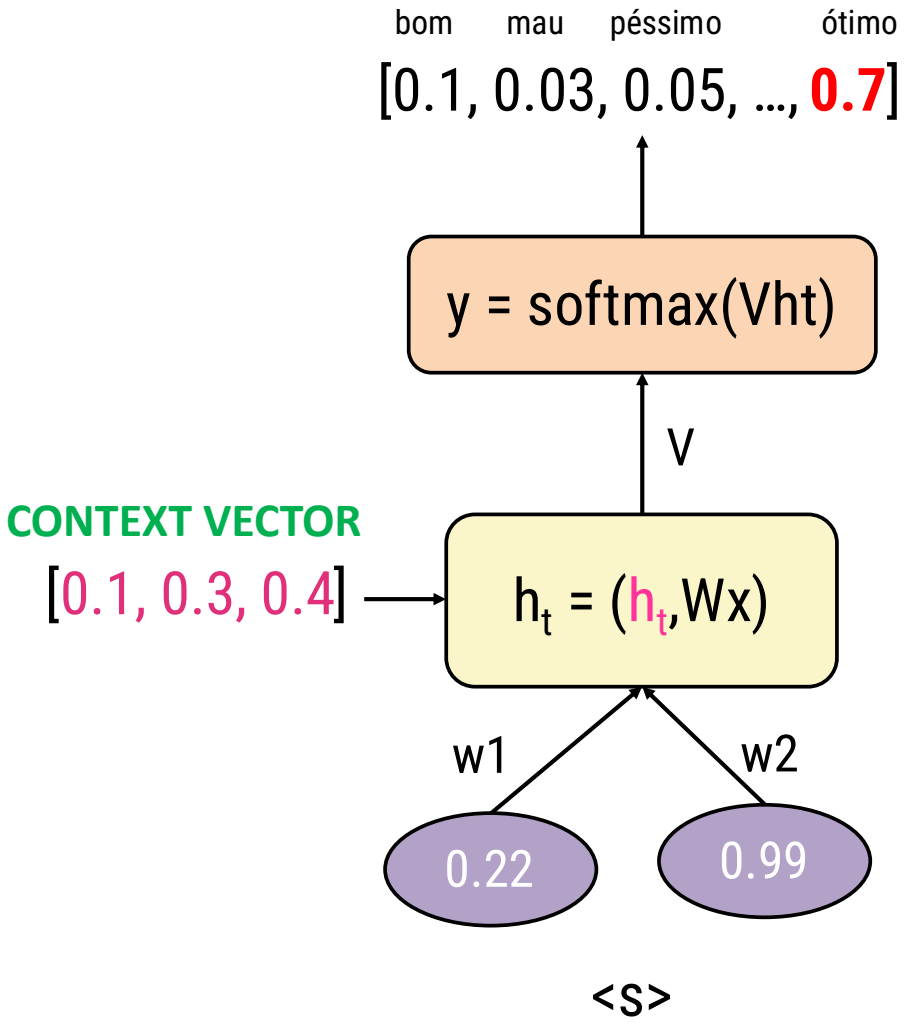


Decoder


y = softmax(Vh_t)
V
CONTEXT VECTOR
[0.1, 0.3, 0.4]
h_t = (h_t,Wx)
w1
w2
0.22
0.99
<s>

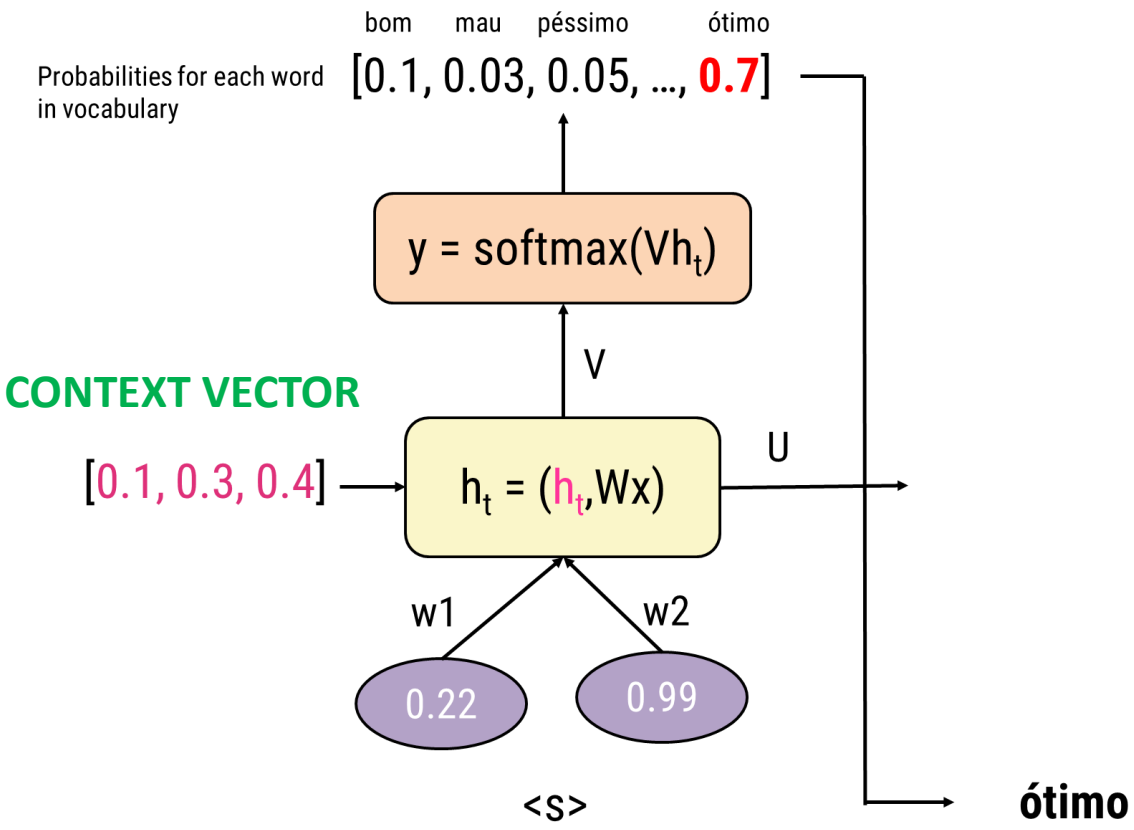

Output layer
Softmax activation function
Probabilities
1.3
5.1
2.2
0.7
1.1
$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$
0.02
0.90
0.05
0.01
0.02

Decoder

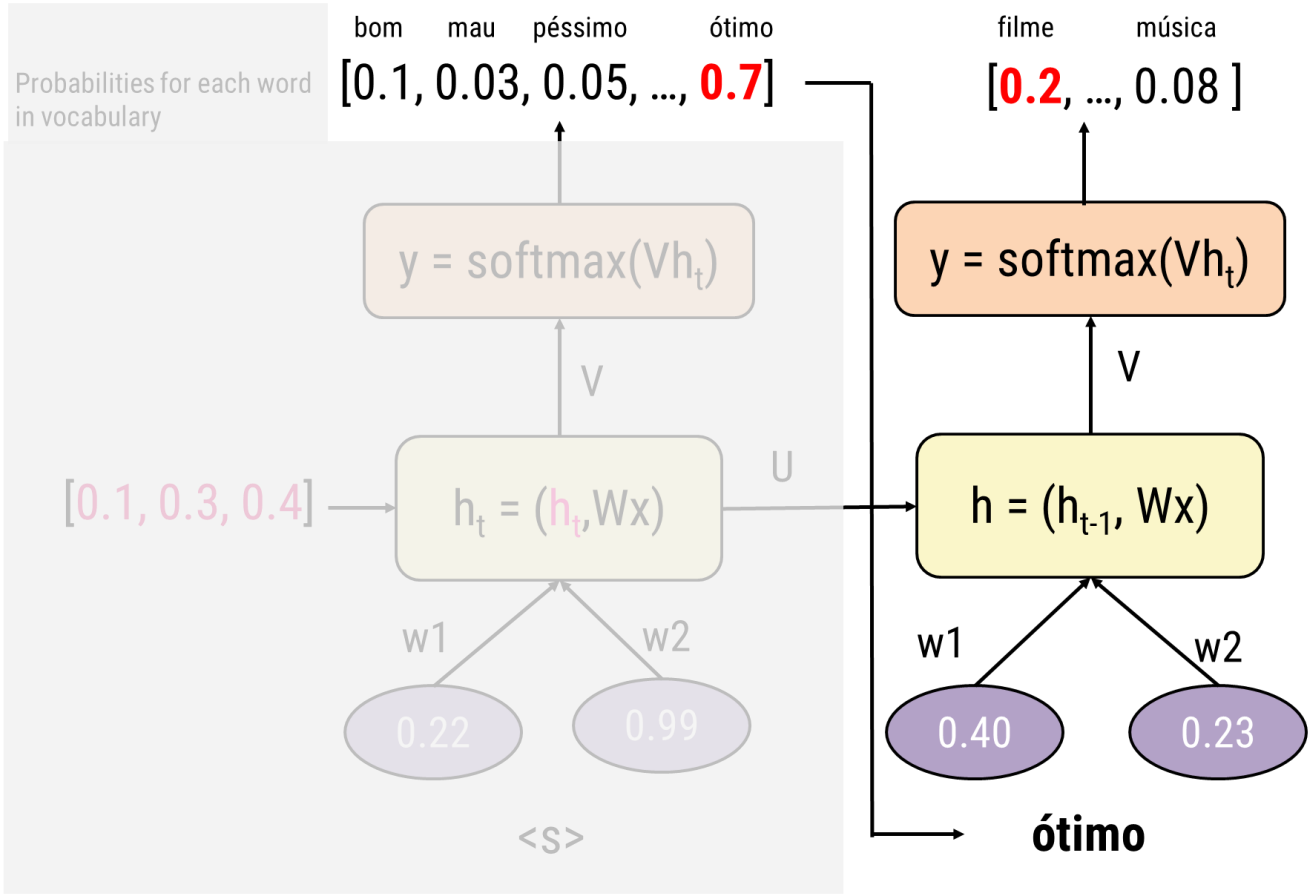


Probabilities for each word in vocabulary

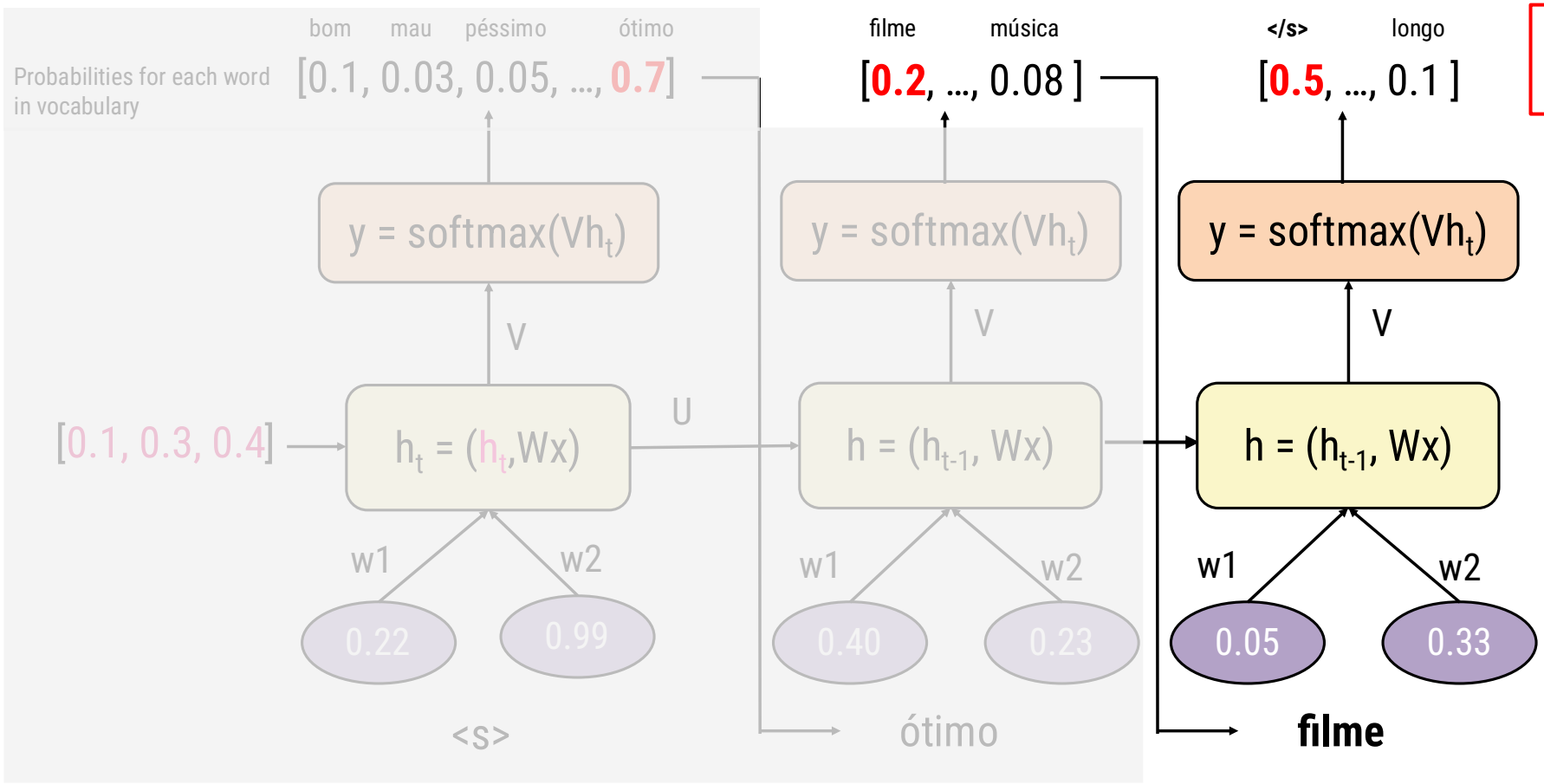
Decoder


bom
mau
péssimo
ótimo
Probabilities for each word in vocabulary
[0.1, 0.03, 0.05, …, 0.7]
y = softmax(Vh_t)
V
CONTEXT VECTOR
[0.1, 0.3, 0.4]
h_t = (h_t,Wx)
U
w1
w2
0.22
0.99
<s>
ótimo

Decoder


bom
mau
péssimo
ótimo
[0.1, 0.03, 0.05, …, 0.7]
Probabilities for each word in vocabulary
y = softmax(Vh_t)
V
[0.1, 0.3, 0.4]
h_t = (h_t,Wx)
U
w1
w2
0.22
0.99
<s>
filme
música
[0.2, …, 0.08]
y = softmax(Vh_t)
V
h = (h_t-1, Wx)
w1
w2
0.40
0.23
ótimo

Decoder


bom
mau
péssimo
ótimo
Probabilities for each word in vocabulary
[0.1, 0.03, 0.05, …, 0.7]
filme
música
[0.2, …, 0.08]
</s>
longo
[0.5, …, 0.1]
After predicting </s> the network stops or moves to the next sentence.
y = softmax(Vh_t)
y = softmax(Vh_t)
y = softmax(Vh_t)
V
V
V
[0.1, 0.3, 0.4]
h_t = (h_t,Wx)
U
h = (h_t-1, Wx)
h = (h_t-1, Wx)
w1
w2
w1
w2
w1
w2
0.22
0.99
0.40
0.23
0.05
0.33
<s>
ótimo
filme

Bibliographic references:

Dan Jurafsky and James H. Martin. **Speech and Language Processing** 3rd ed.

Chapter 5

https://web.stanford.edu/~jurafsky/slp3/

Colah's Blog:
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Michael Phi:
https://medium.com/data-science/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21

NOVA IMS INFORMATION MANAGEMENT SCHOOL

Thank you!

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · apcer · IQNet

Cofinanciado por

PORTUGAL 2030 · 2030 Lisboa Programa Regional de Lisboa · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU